

Réseaux de capteurs et IOT

Dr Ida TOGNISSE

idatognisse@gmail.com

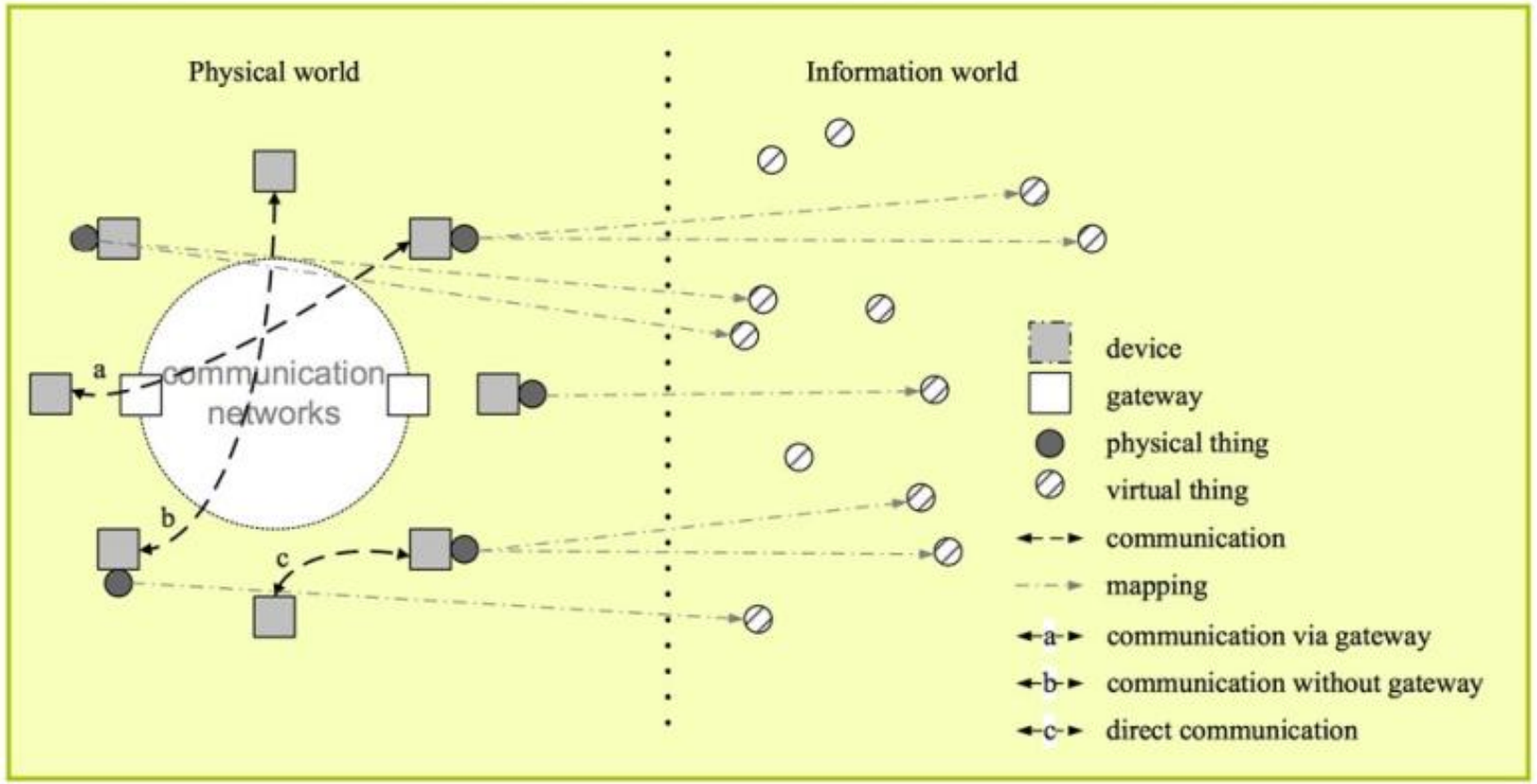
Présentations

- Nom
- Background
- Domaine d’interets
- Attentes

Objectif général

L'objectif général de cette unité d'enseignement vise à permettre à l'apprenant d'utiliser les architectures adéquates pour le déploiement des capteurs et objets. Les apprenants doivent utiliser les protocoles de l'IoT et réaliser des choix compte tenu des besoins pour les applications portées par les objets choisis.

"Les Objets **physiques** existent dans le monde physique et sont capables d'être **détectées**, **actionnées** et **connectées**. Le milieu environnant, les robots industriels, les marchandises et les équipements électriques sont des exemples d'objets physiques."



Définitions

• **Objet connecté :** objet possédant la capacité d'échanger des données avec d'autres entités physiques ou numériques.

• **Internet des objets (IdO) :** expansion du réseau internet à des objets et/ou des lieux du monde physique. En anglais, on parle d'**IoT : Internet of Things**. « L'Internet des Objets est un réseau de réseaux qui permet, via des systèmes d'identification électronique normalisés et unifiés, et des dispositifs mobiles sans fil, d'identifier directement et sans ambiguïté des entités numériques et des objets physiques et ainsi de pouvoir récupérer, stocker, transférer et traiter, sans discontinuité entre les mondes physiques et virtuels, les données s'y rattachant.

Définitions

"L'IoT peut être considéré comme une infrastructure mondiale pour la société de l'information, permettant des services avancés en interconnectant des objets **(physiques** et **virtuels)** sur la base des technologies de l'information et de la communication (TIC) interopérables existantes et en évolution."

L'IoT est l'ensemble des objets (capteurs et actuateurs) repartis géographiquement et leur interconnexion avec l'internet. Ces objets (capteurs) collectent des masses de données hétérogènes et les envoient à des serveurs d'application à travers une connectivité pour leur analyse en vue d'une prise de décision.

Le concept d’ « objet connecté » n’est pas nouveau

- Le premier système de télémétrie a été mis en place à Chicago en **1912**. On dit qu'il utilisait des **lignes téléphoniques** pour surveiller les données des centrales électriques. La télémétrie s'est étendue à la surveillance météorologique dans les **1930**, lorsqu'un dispositif connu sous le nom de **radiosonde** est devenu largement utilisé pour surveiller les conditions météorologiques depuis des ballons.
- L'adoption à grande échelle de la technologie M2M a commencé dans les années 1980 avec les connexions câblées pour le **SCADA** (contrôle de surveillance et acquisition de données) dans les usines.
- Dans les années 1990, ADEMCO a construit son propre **réseau radio privé** car la connectivité cellulaire était trop coûteuse.
  En 1995, Siemens a lancé le **premier module cellulaire** conçu pour le M2M

Le concept d’ « objet connecté » n’est pas nouveau



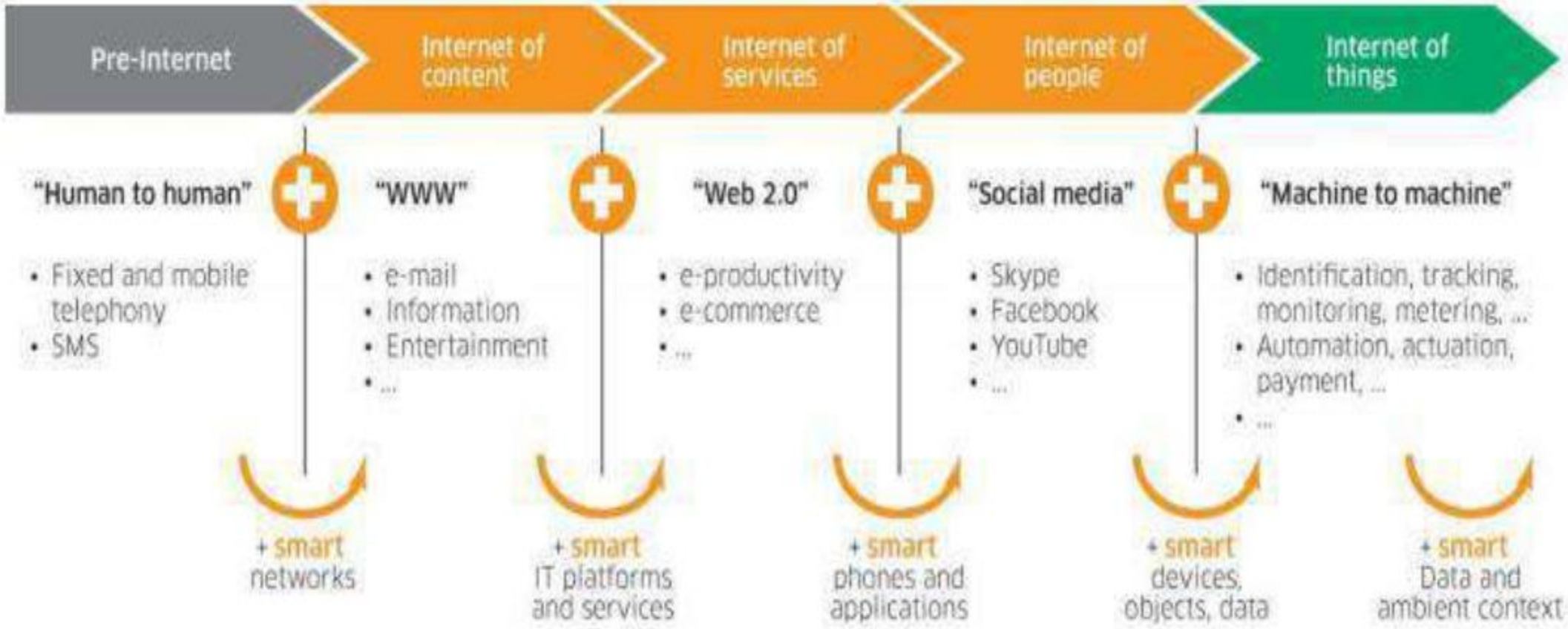
caméra pointée sur une cafetière et connectée au réseau local de l’université Cambridge en 1991



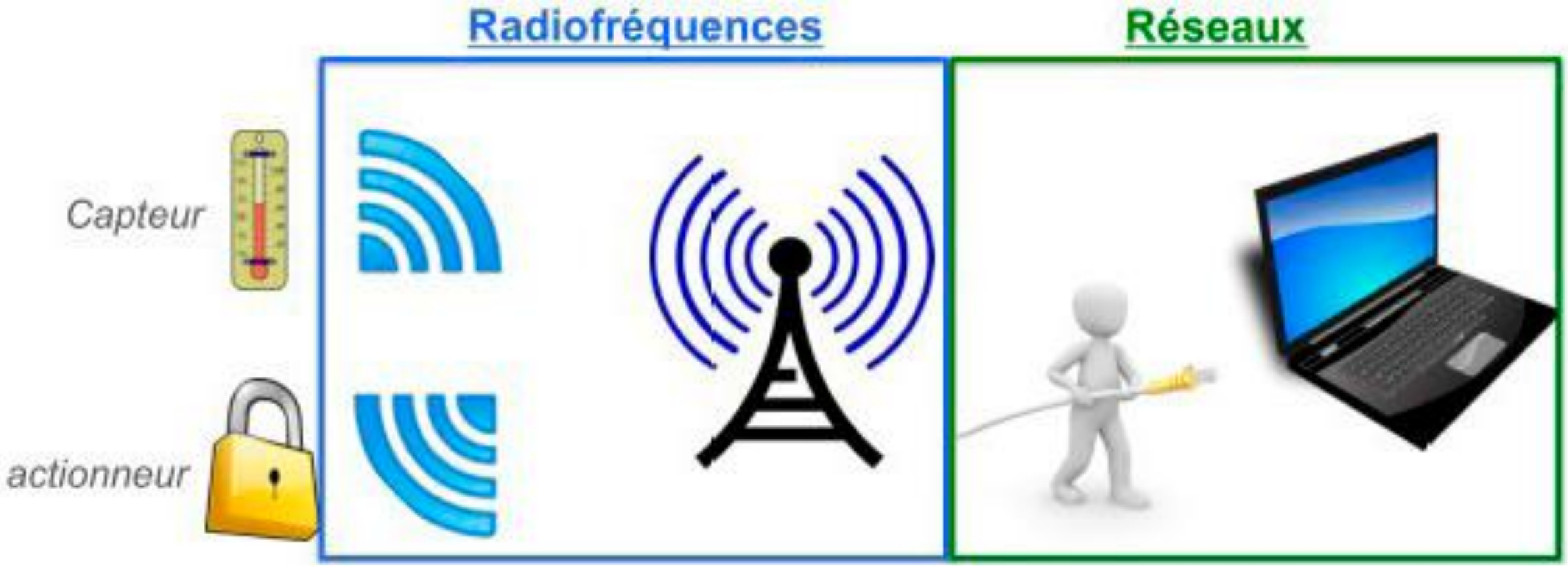
Ce lapin connecté en Wi-Fi peut déjà lire des mails à haute voix, émettre des signaux visuels et diffuser de la musique. L’objet est toujours commercialisé aujourd’hui, sous le nom de **Karotz**, Lancé par la société Violet en 2005



Telegarden : juin 1995 (university of California)


Pre-Internet
Internet of content
Internet of services
Internet of people
Internet of things
"Human to human"
• Fixed and mobile telephony
• SMS
+ smart networks
"WWW"
• e-mail
• Information
• Entertainment
• ...
+ smart IT platforms and services
"Web 2.0"
• e-productivity
• e-commerce
• ...
+ smart phones and applications
"Social media"
• Skype
• Facebook
• YouTube
• ...
+ smart devices, objects, data
"Machine to machine"
• Identification, tracking, monitoring, metering, ...
• Automation, actuation, payment, ...
• ...
+ smart Data and ambient context

Une solution connectée est constituée par des objets connectés (capteurs et actionneurs) et par un réseau avec une partie radiofréquence pour les communications sans fil et une autre partie qui utilise Internet (qui constituent le réseau à proprement parler).



- Quand on parle de liaison par radiofréquence, la première caractéristique à identifier est la fréquence de ses ondes RF. Les fréquences RF ne sont pas toutes librement utilisables ou sans contrainte. Ainsi les opérateurs doivent acheter des bandes de fréquences à l'État.
- L'émission peut être unidirectionnelle comme dans le cas d'un capteur de température qui envoie seulement une donnée, ou bidirectionnelle dans le cas d'un téléphone portable.
- Une autre caractéristique des objets connectés est sa portée. La portée est la distance à laquelle les ondes ne sont plus captables par les objets. Cette portée définit alors un disque de couverture dans lequel les objets peuvent être adressés. Le disque de couverture radio s'appelle une cellule. Pour former un réseau cellulaire, il faut considérer que la cellule est octogonale. Ainsi, six cellules contiguës vont être pilotées par une station de base et former un réseau.
- Pour que ce système cellulaire fonctionne, il faut que chaque cellule opère à des fréquences RF différentes pour éviter les brouillages en bord de cellule. Évidemment, il faut également une coordination des stations de base.

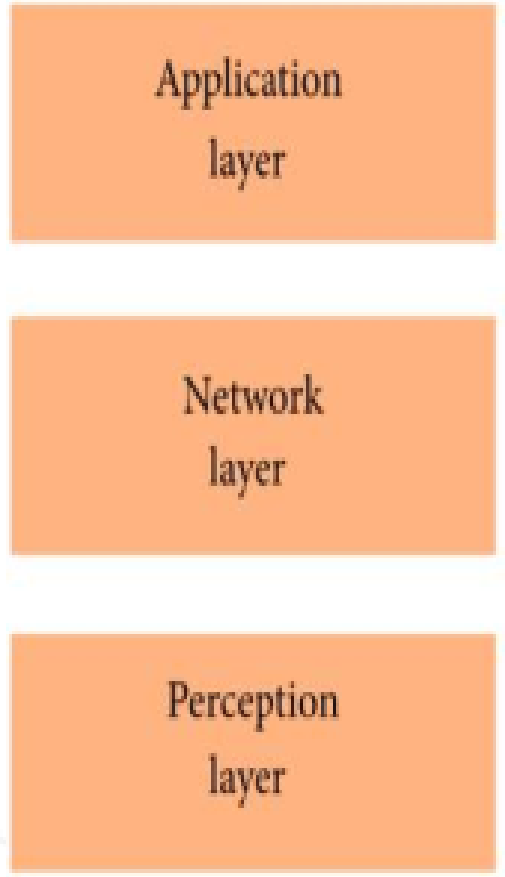
• L‘architecture d‘une solution IoT varie d‘un système à l‘autre en se basant sur le type de la solution à mettre en place.

• L’architecture la plus élémentaire est une architecture à trois couches:

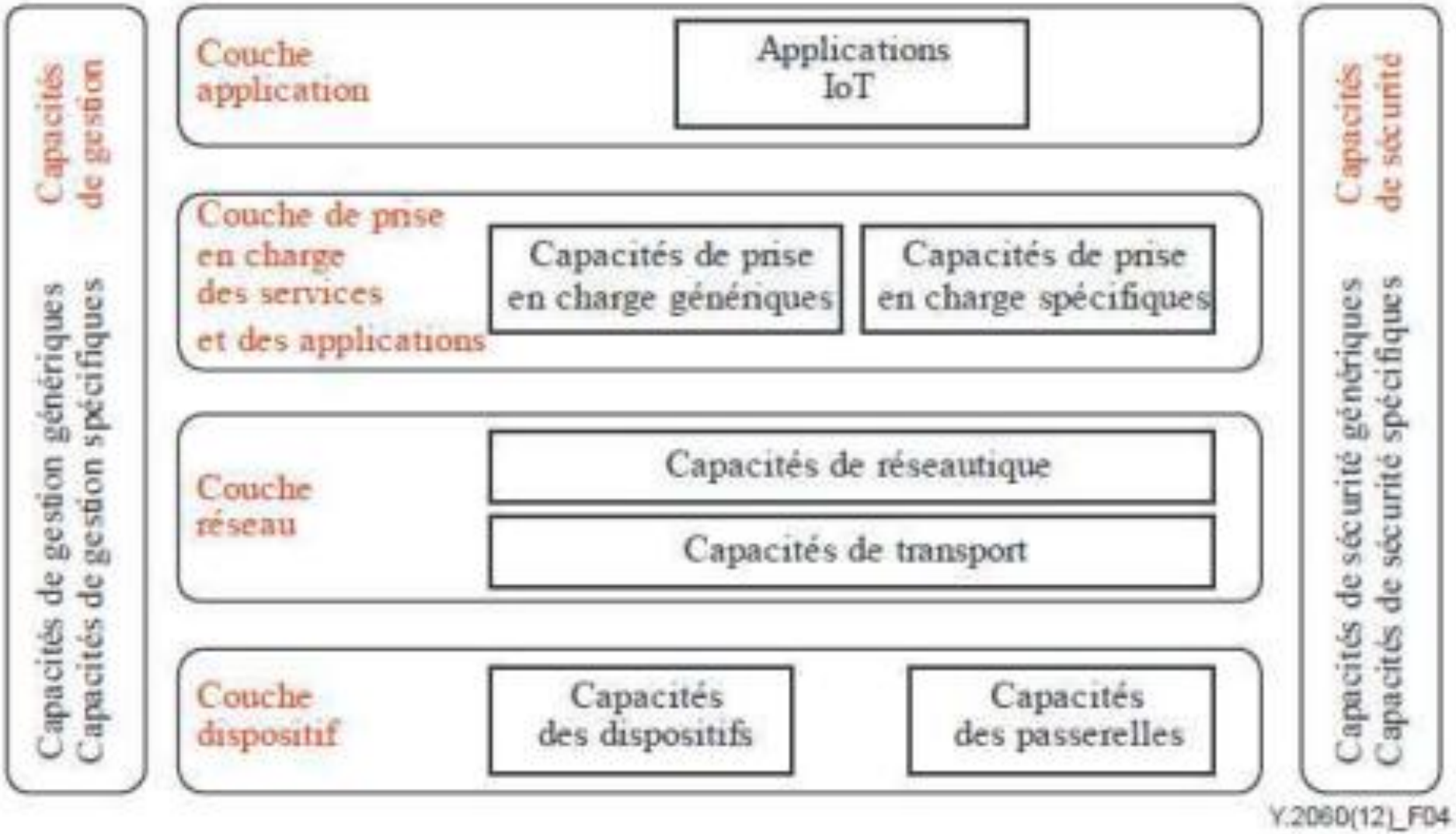
– **La couche perception** possède des capteurs et actionneurs qui détectent et recueillent des informations sur l’environnement.

– **La couche réseau** est responsable de la connexion, du transport et du traitement des données issues des capteurs et actionneurs.

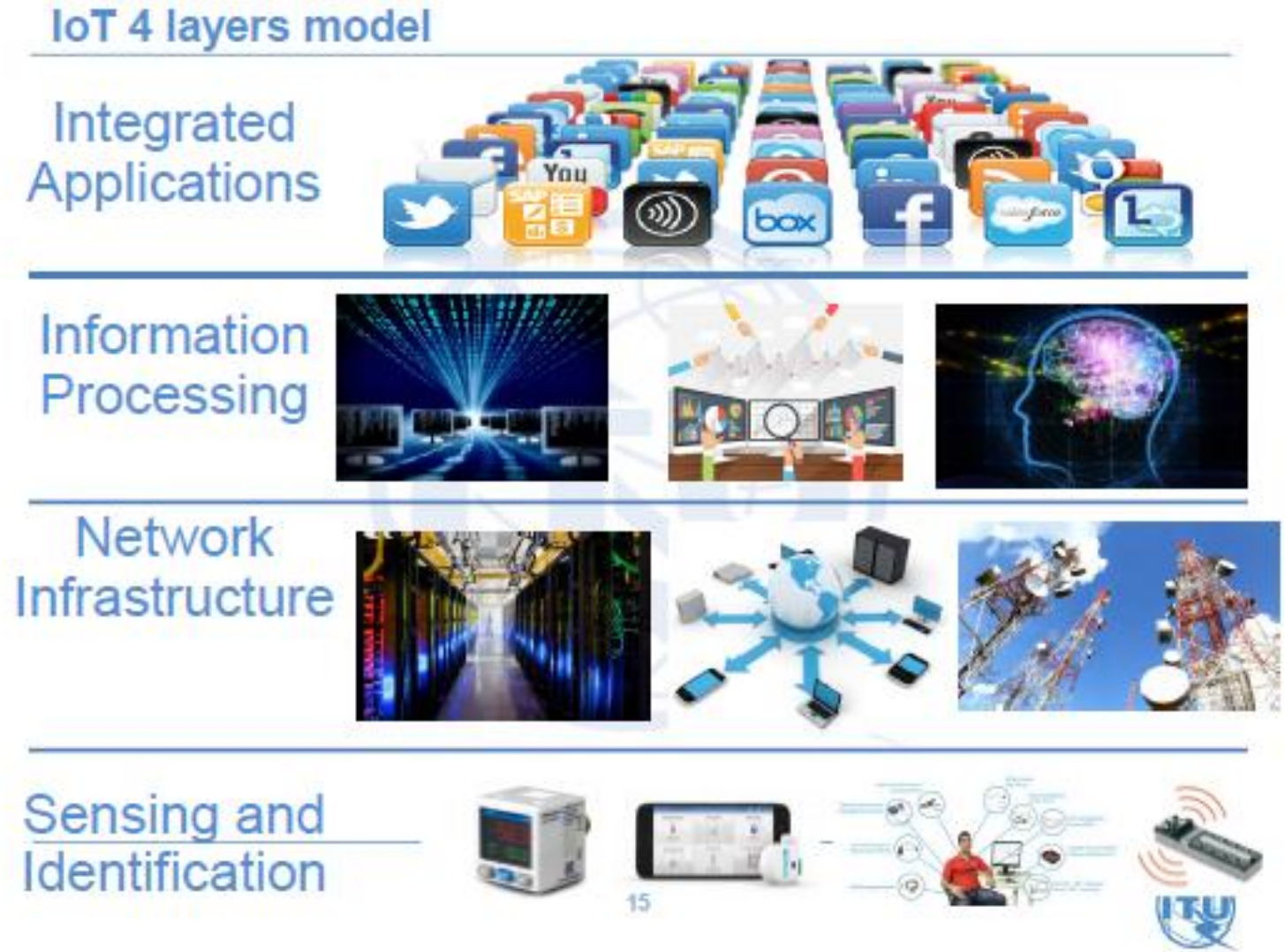
– **La couche application** est chargée de fournir à l’utilisateur des services spécifiques et applications intelligentes.



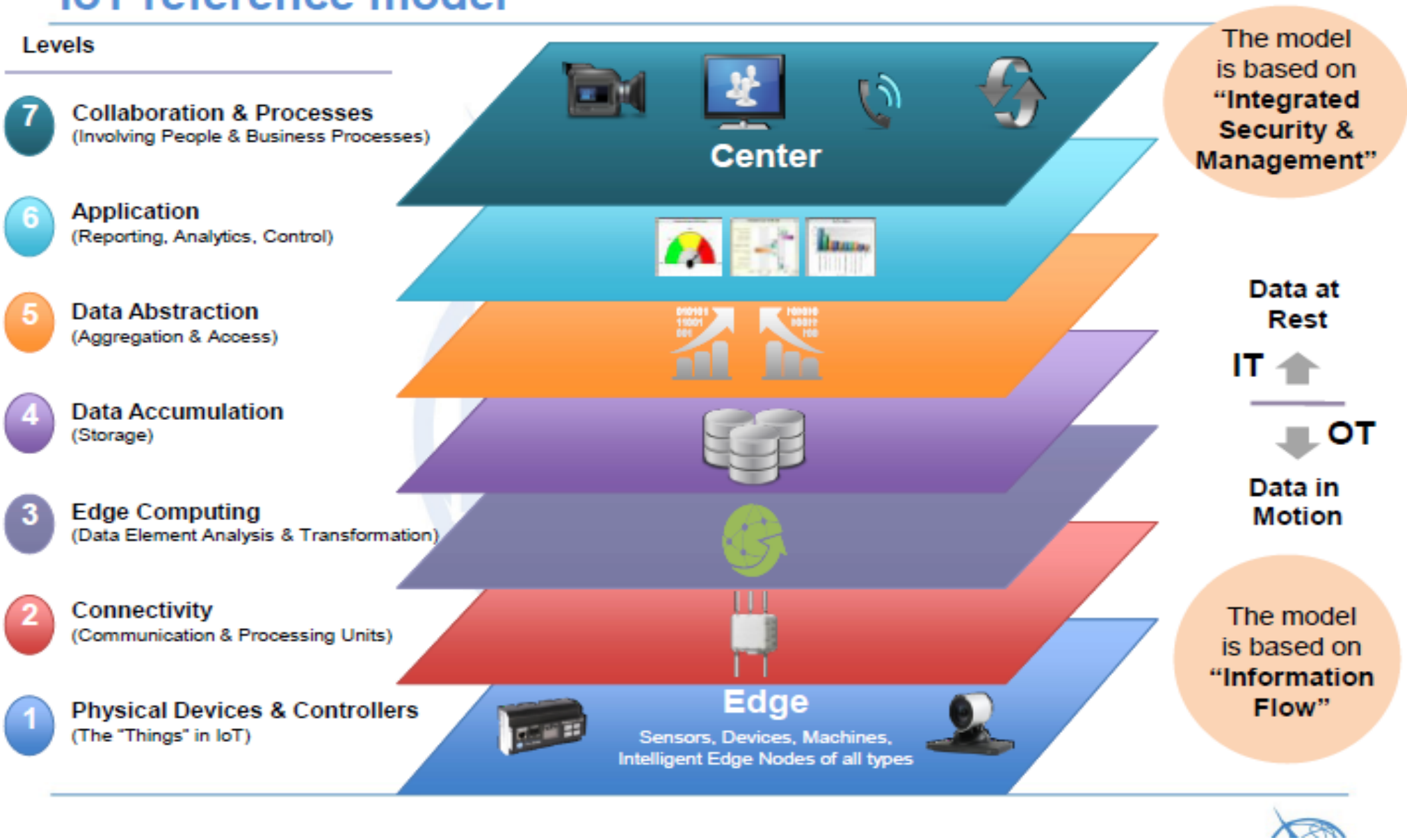
- Modèle de référence de l'IoT selon la recommandation UIT-T Y.2060



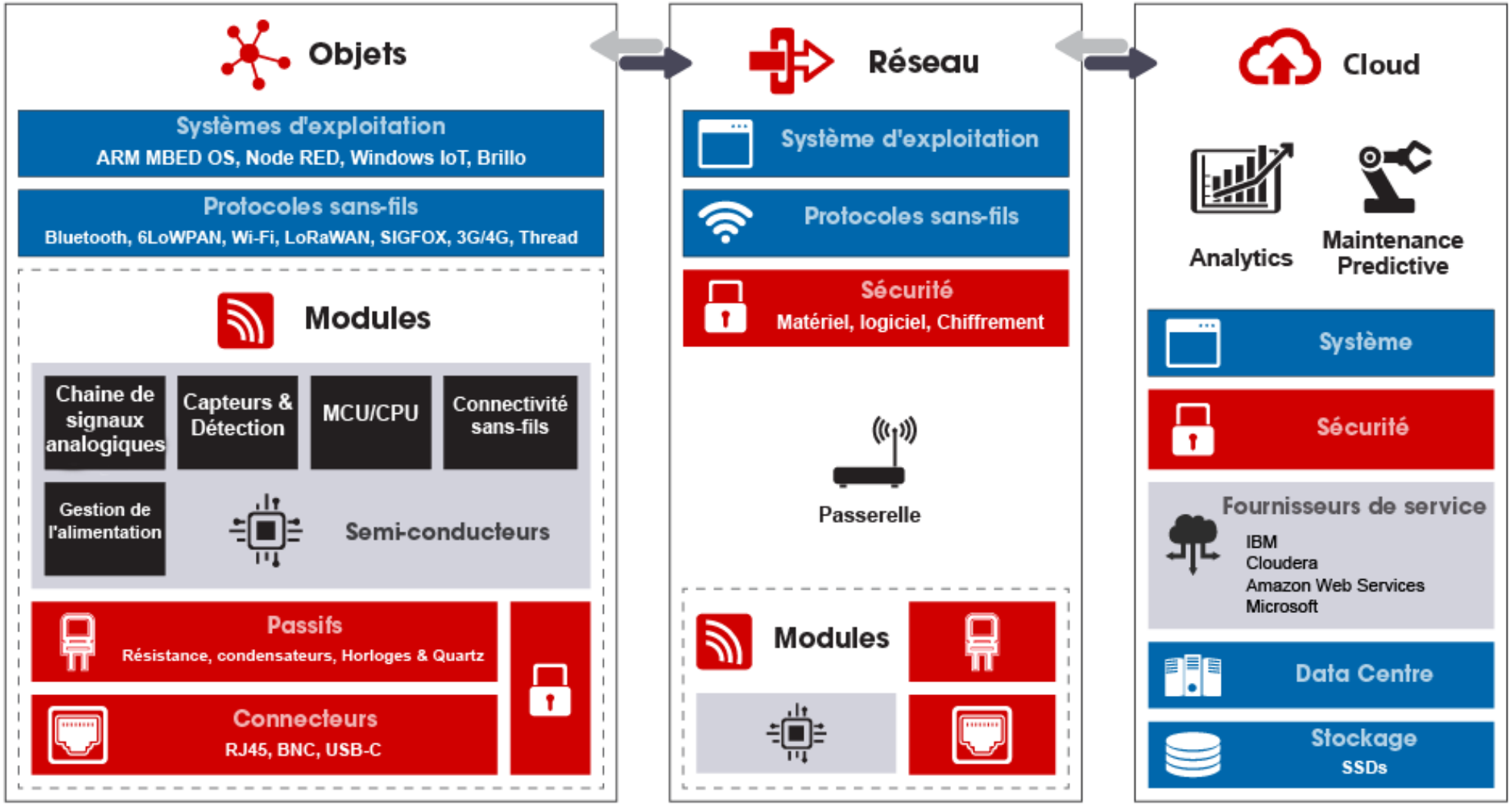
- Modèle de référence de l'IoT selon la recommandation UIT-T Y.2060



IoT reference model

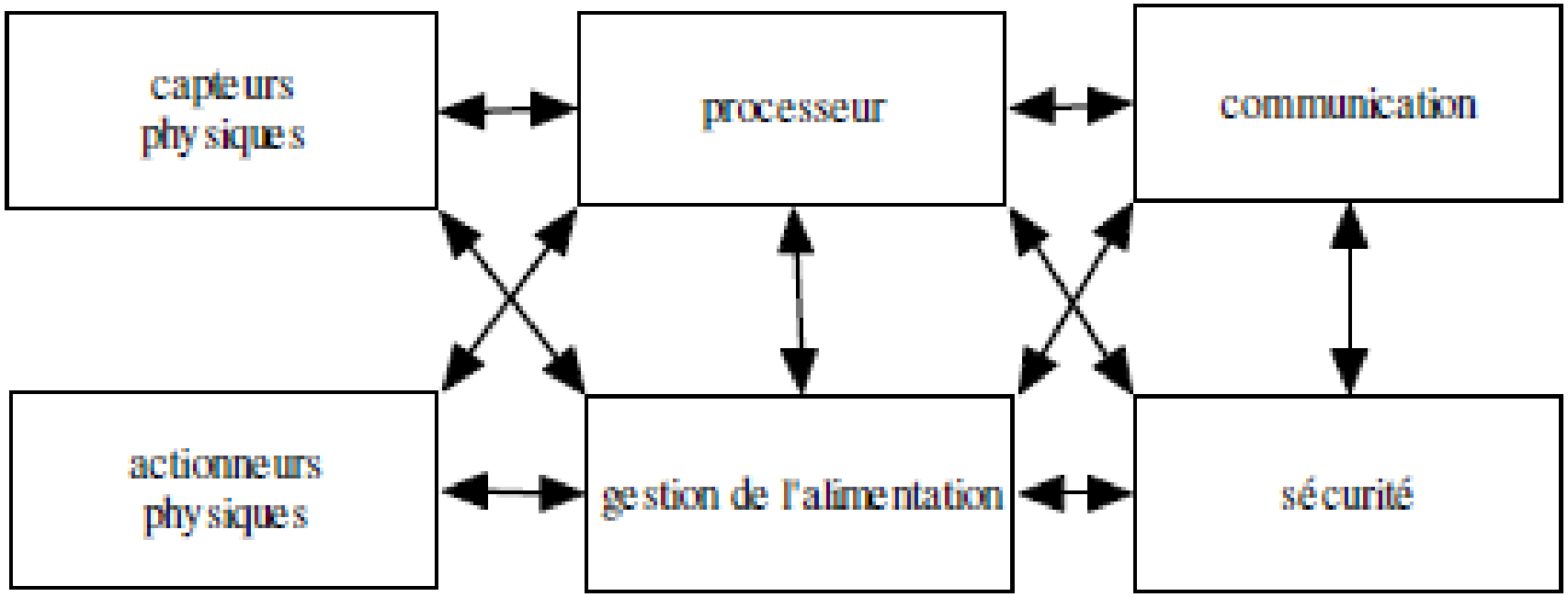


Architecture fonctionnelle d'une solution IoT



Un capteur sans fil : Un capteur est essentiellement composé de

- Unités de capture
- Unité de traitement (Processeur et mémoire)
- Unité de transmission (Émetteur/récepteur)



Composants d’un capteur

• **Processeur** : faible consommation, vitesse d'horloge faible (<400MHz), puissance de calcul faible (300 MIPS).
• **Mémoire** : RAM (qques dizaines de Mo) + EEPROM (pour programme embarqué) + Flash (de l'ordre du Go)
• **Connecteurs** : USB, série, cartes d'extension (ethernet, …)
• **Périphs** : timers, lignes d'E/S, PWM, CA/N, horloge
• **Capteurs** :
– Environnement : pression, humidité, lumière, température …
– Force et déplacement : rotation, position, accélération
– Electromagnétique : antenne, magnétisme
– Chimique : composition de gaz
– Autres : son, image, infrarouge
Constitution : capteur + conditionnement de signal + conversion A/N + DMA parfois
Possibilité de mise en veille pour consommation
• **Actionneurs** : LEDS, buzzer, optocoupleurs pour connexions externes …
• **Alimentation** : par batteries rechargeables + régulateur + contrôle de niveau de batterie + contrôle de mise en veille et de vitesse d'horloge (baisse pour baisser la conso) et de tension d'alim du processeur (mode faible conso). Parfois ajout de chargement par solaire ou vibration ou vent …
Remarque : le nombre de cycles de charge / décharge est limité.

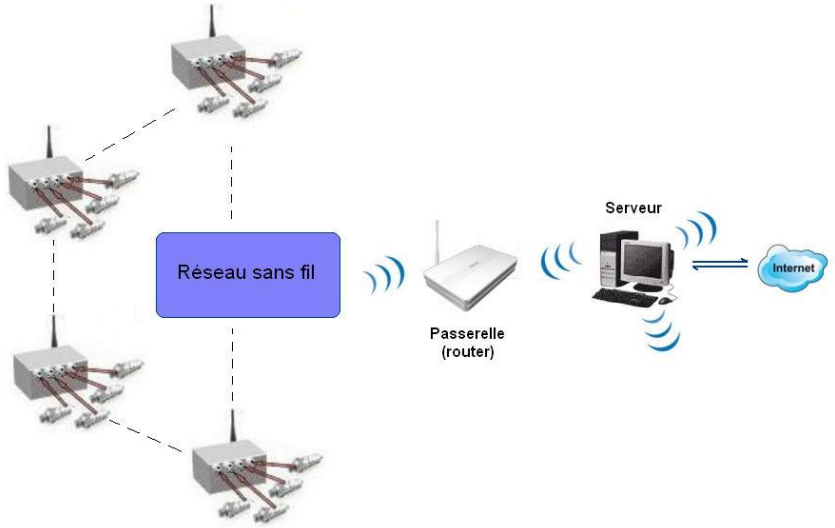
Un RCSF (Wireless Sensor Networks (WSN)) est un ensemble de capteurs regroupés au sein d'un même réseau sans fil.

- considéré comme un type spécial de réseaux ad hoc
- composé de « noeuds »

notion de « champs de captage ».

protocole de routage : étoile, « toile » ou hybride.

« point de collecte » requêtes utilisateur



Description du RCSF

Les capacités de traitement et de mémorisation des capteurs leur permettent de jouer le rôle d'un noeud actif dans un réseau relativement large. Lorsque le nombre de capteurs devient conséquent, la communication en réseau devient indispensable.

•Il n'est en effet alors plus possible d'atteindre un capteur directement par un câble ou même par une connexion radio dans un réseau très large.

•La mise en place d'un réseau de capteurs capable de s'autoconfigurer et de s'auto organiser de façon dynamique s'impose.

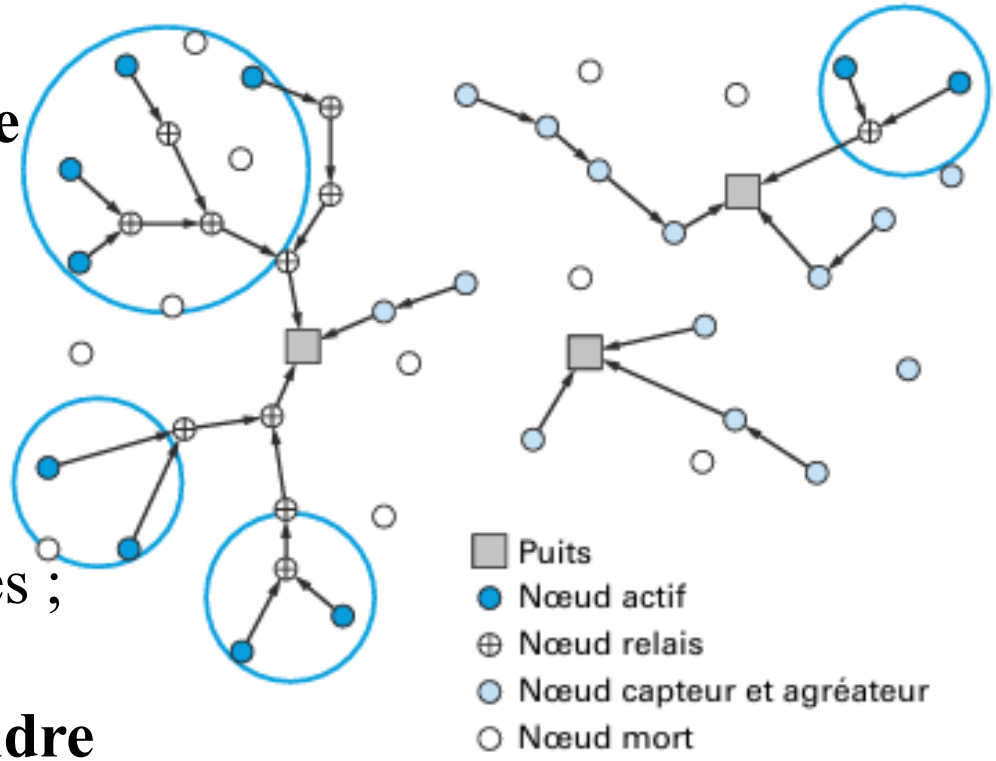
• Un réseau de capteurs se compose de deux types de noeuds : **des simples capteurs** et des collecteurs d'informations appelés **puits.**

Le puits est un élément très important d'un réseau de capteurs.

Il possède des capacités supérieures en termes de puissance de traitement, de capacité de mémoire et d'autonomie d'énergie. Elle permet de collecter l'information en provenance des capteurs et apporte un soutien très fort au fonctionnement du réseau.

Elle peut localement assurer des fonctions centrales dans le routage, l'agrégation des données, la configuration des noeuds ou encore l'organisation de l'ordre de transmission et de réception des différents capteurs avoisinants.

La figure présentée sur ce slide illustre un exemple de réseau de capteurs où nous pouvons distinguer différents scénarios possibles.
Sur cette figure, nous pouvons repérer :
des puits



• des noeuds hors service car leurs batteries sont usées ;
• des noeuds actifs en étant soit source de l'information ou servant comme relais pour atteindre le puits de la collecte d'informations ;
• des noeuds endormis qui se trouvent dans leur état oisif.

Les noeuds actifs

Les noeuds actifs sont équipés de capteurs pour mesurer divers paramètres environnementaux. Ils collectent, traitent et transmettent les données de l'environnement à l'aide de leurs capteurs. Ils sont responsables de la surveillance continue et de la collecte des données. Les noeuds actifs transmettent l'information qu'ils détiennent. Lors de la transmission, le noeud doit s'assurer qu'il est seul à occuper l'espace de transmission afin d'éviter des interférences sur les autres transmissions. **Ces noeuds sont présentés en couleur bleu sur la figure.**

Les noeuds relais

Les noeuds relais sont des noeuds indispensables dans un réseau large. En effet, il est impossible de couvrir tous les noeuds du réseau par une seule transmission. L'atténuation des signaux radio et d'autres phénomènes comme l'évanouissement ou l'effet multi-trajet font que le signal, à partir d'une certaine distance, peut se dégrader et contenir un nombre important d'erreurs le rendant incompréhensible. Pour cette raison, il faut relayer l'information. Ceci permet de la récupérer dans un noeud intermédiaire, de la régénérer avec une énergie remise à neuf redonnant ainsi à l'information la possibilité d'atteindre une destination plus lointaine (le puit).

Les noeuds agrégateurs

Les noeuds agrégateurs sont des noeuds placés stratégiquement dans le réseau pour optimiser la collecte et le traitement des données des noeuds capteurs. Ils captent la grandeur physique et la gardent afin d'agréger l'information mesurée qui sera envoyée plus tard pour réduire la consommation d'énergie. Il collecte les données provenant de plusieurs noeuds capteurs, les traite et les agrège pour les transmettre de manière plus efficace vers une station centrale. Un noeud agrégateur joue un double rôle en collectant et en traitant les données tout en agissant comme un relais pour acheminer ces données vers la destination finale. **Ces noeuds sont présentés en couleur bleu clair sur la figure.**

Conditions d'optimisation du fonctionnement des réseaux de capteurs

- Toutes ces opérations sont organisées au sein des réseaux de capteurs dans le but d'optimiser le fonctionnement du réseau:
- Endormir et réveiller les noeuds
- Accéder à la transmission sans collision ni interférence avec les voisins
- Acheminer l'information
- Gérer l'énergiex de façon à réduire le nombre de capteurs qui disparaissent

Contraintes liées à la conception d'un réseau de capteurs

- Contraintes liées à l'application
- Contrainte énergétique
- Contraintes liées aux déterminismes
- Contraintes de passage à l'échelle
- Contraintes liées à la qualité de service
- Contraintes liées à la protection de l'information
- Contraintes liées à l'environnement

3 solutions pour les RCSF:

- Technologies dédiées aux réseaux sans fil (IEEE)
- Téléphonie mobile
- Communication par satellites

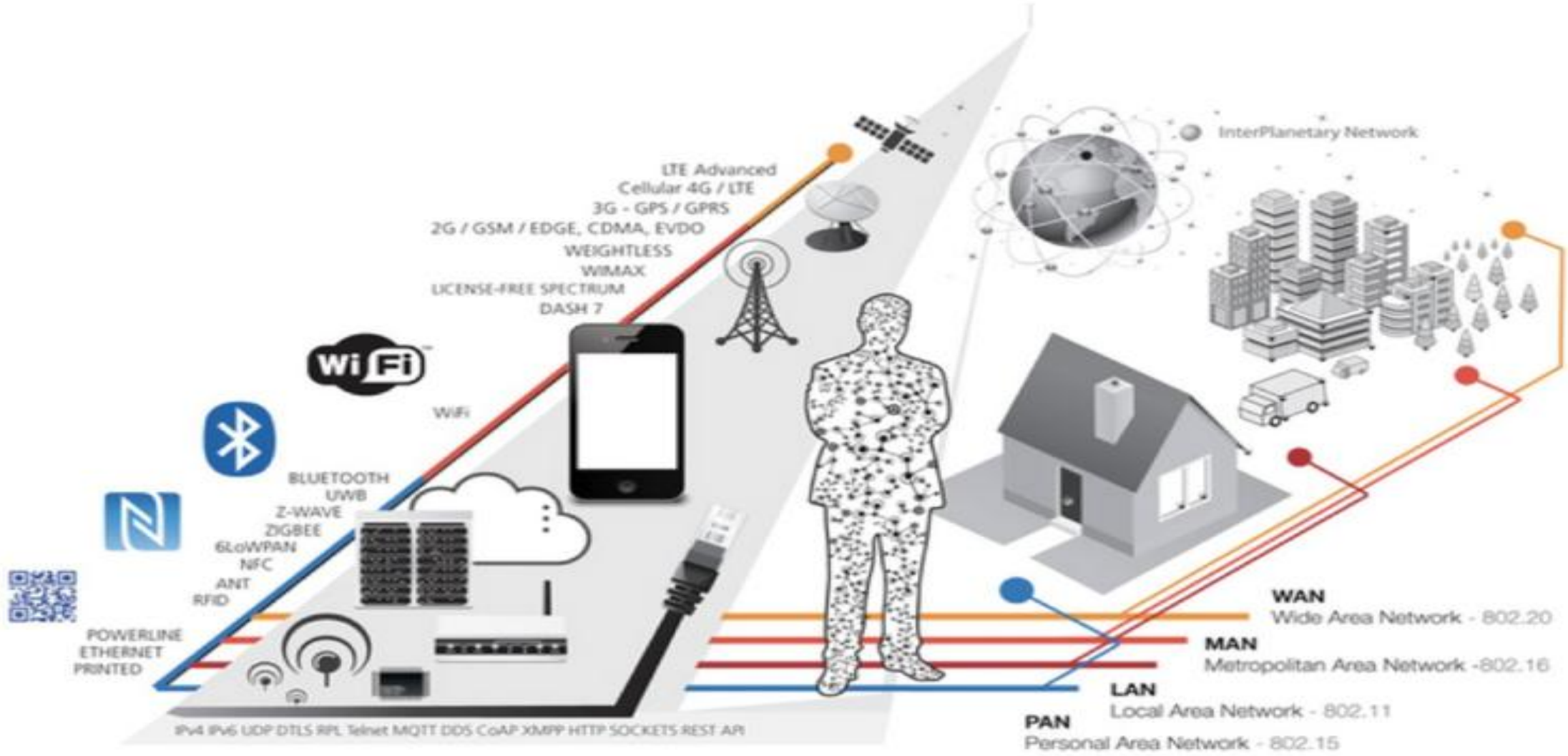
Contraintes :

- Problèmes d'atténuation (distance, obstacles, pluie)
- Problèmes de Multi-trajet
- Problèmes d'interférences

Connectivité

- Capacité à **communiquer** avec un autre ordinateur.
- Réseau personnel (PAN, par exemple BLE, Zigbee).
- Réseaux locaux (LAN, par exemple Ethernet, WiFi).
- Réseaux étendus (WAN, par exemple 3/4G, LoRaWAN).
- La portée s'étend de "pièce" à "bâtiment" et à "ville". (par exemple, BLE, 30 m ; Wi-Fi, 100 m ; LoRaWAN, 2-15 km).

Connectivité



Connectivité : aspects clés

- **Portée** - êtes-vous en train de déployer un seul étage de bureau ou une ville entière ?
- **Débit de données** - de quelle bande passante avez-vous besoin ? Combien de fois vos données changent-elles souvent ?
- **Alimentation** - votre capteur fonctionne-t-il sur secteur ou sur batterie ?
- **Fréquence** - avez-vous pris en compte le blocage des canaux et l'interférence des signaux ?
- **Sécurité** - vos capteurs prendront-ils en charge des applications applications critiques ?

Gestion de mémoire

Utilisation de mémoire moire de type flash

Adressage physique et logique sous TFFS

Ne représente pas un problème aujourd'hui dans les RCSF mais influe sur la stratégie énergétique

Réflexion sur la capacité de stockage en fonction de l'utilisation finale du produit (fréquence d'acquisition, volume de données etc.).

En agri. : généralement, stockage limité de données brutes.

Technologies clés génériques

L‘IoT fonctionne avec le support de plusieurs technologies tels que les réseaux de capteurs sans fil, le Cloud Computing, les analyses Big Data, les protocoles de communication, les services web, etc

Les réseaux de capteurs sans fil RCSF: (Wireless Sensor Network, WSN) Un RCSF se compose d‘un nombre de Noeuds-Capteurs qui ont des fonctionnalités de capturer et traiter/transmettre les données.

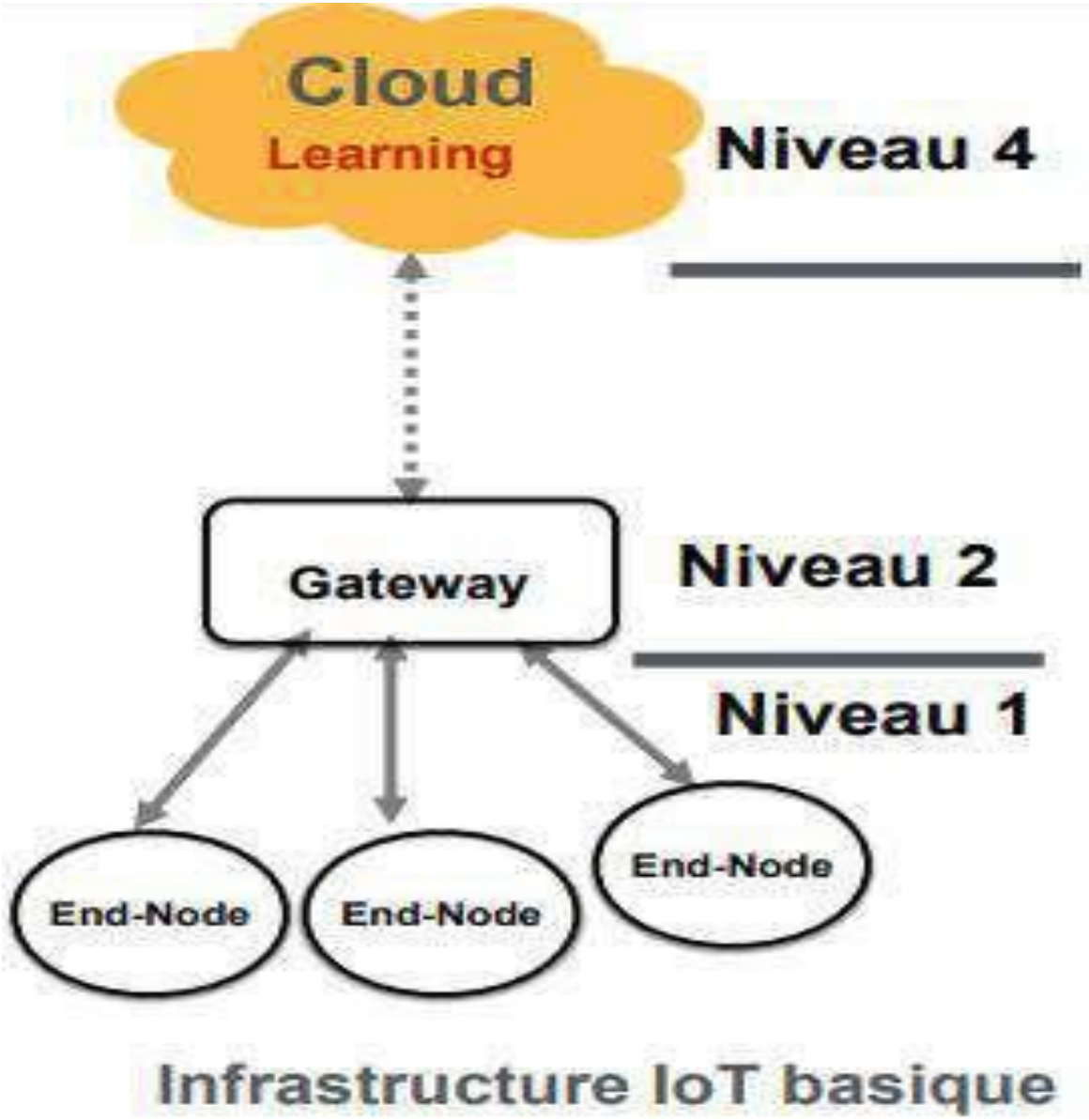
Cloud Computing : fournit un espace de stockage de données IoT et offre des services de visualisation, analyse et archivage des données.

Big Data : offre des outils d‘analyse avancées pour les données massives collectées par les objets IoT selon leurs caractéristiques : volume, vitesse, variabilité (forme de données : texte, audio, video, image).

– Les protocoles de communication : sont indispensables pour assurer la connectivité entre objets et applications. Les protocoles de communication définissent le format des données, taille paquets, adressage, routage, etc.
– Les systèmes embarqués : Les objets connectés sont formés essentiellement des cartes à microcontrôleur intégrant un microprocesseur, une mémoire et des ports d‘ E/S pour la connexion des capteurs

Cloud Computing

3 types de cloud IoT
– IaaS: Infrastructure as a Service
– PaaS: Plateforme as a Service
– SaaS: Software as a Service

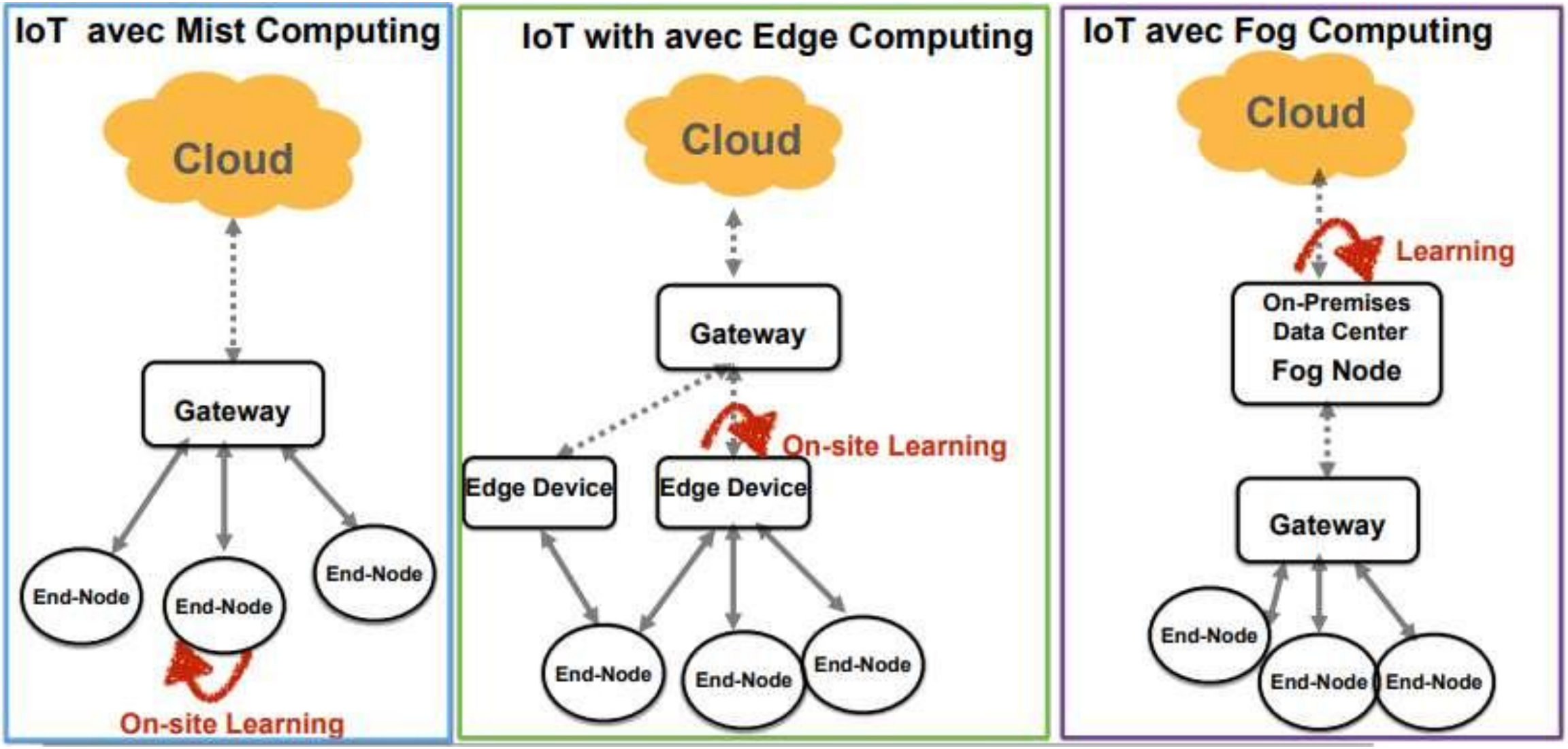


Trois solutions techniques sont possibles

– Fog Computing : permet un calcul décentralisé en traitant les données IoT au niveau des nœuds locaux avant de relayer l'information vers le cloud.

– Edge Computing : le traitement des données IoT se fait à l'extrémité du réseau (Gateways ou des noeuds intermédiaires entre objets et gateways).

– Mist Computing : le traitement des données se fait localement dans le noeud capteur.


IoT avec Mist Computing
Cloud
Gateway
End-Node
End-Node
End-Node
On-site Learning
IoT with avec Edge Computing
Cloud
Gateway
On-site Learning
Edge Device
Edge Device
End-Node
End-Node
End-Node
IoT avec Fog Computing
Cloud
Learning
On-Premises
Data Center
Fog Node
Gateway
End-Node
End-Node
End-Node

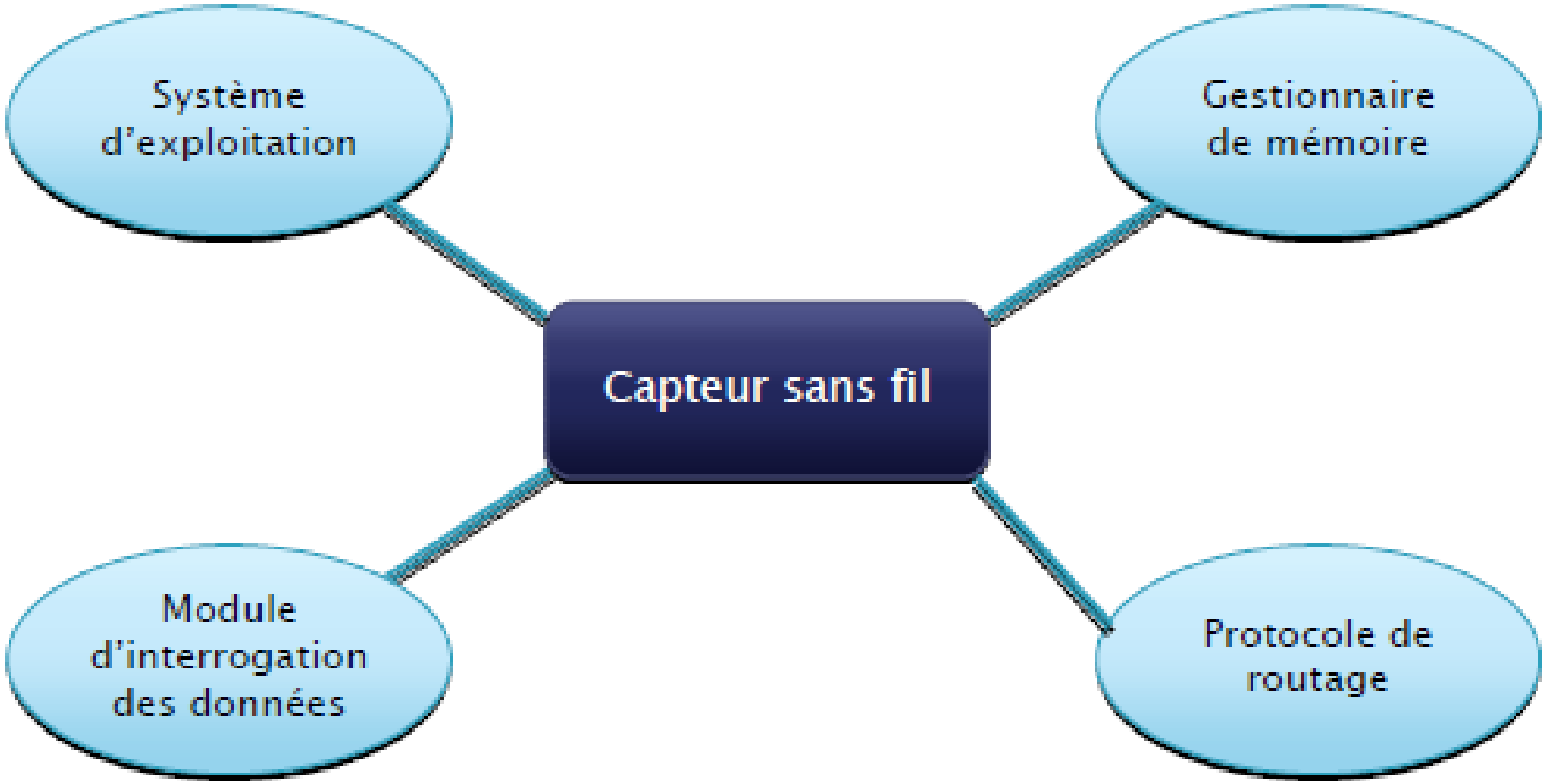
• **Avantages de l’architecture Edge**

– Le déploiement d‘une architecture distribuée repose sur l’idée d’intégration de l‘intelligence près des nœuds capteurs, on the « Edge », ce qui diminue par conséquence la dépendance au cloud computing.

– L’idée est de rapprocher le plus possible le traitement des données et la prise de décisions des nœuds capteurs et réduire ainsi les temps de latence résultant de l’envoi des données au cloud.

– Le traitement « Edge » des paquets de données brutes permet de renforcer la sécurité localement avant de les relayer au cloud.

Composants logiciels



Ingénierie et conception des systèmes IoT

Ingénierie et conception des systèmes IoT

- Analyse d'un besoin réel et rédaction du cahier des charges d'une solution IoT
- Décomposition fonctionnelle : nœuds, passerelle, serveur, stockage et supervision.
- Choix des capteurs, microcontrôleurs, liaisons et contraintes de fonctionnement.
- Diagrammes d'architecture et critères d'acceptation.

On souhaite:

- **concevoir un système de surveillance de l'humidité d'une parcelle agricole au Bénin.**
- **réaliser l'architecture fonctionnelle et technique du système, avec schéma des échanges et liste des composants.**

Pour ressouder ce problème il faut:

- traduire un besoin réel en exigences techniques vérifiables ;
- décomposer un système IoT en blocs fonctionnels ;
- choisir les composants matériels et logiciels adaptés ;
- définir les données échangées entre les composants ;
- produire un schéma d'architecture et un plan de tests ;
- identifier les contraintes de coût, d'énergie, de connectivité et de sécurité.

Définition du problème et analyse du besoin

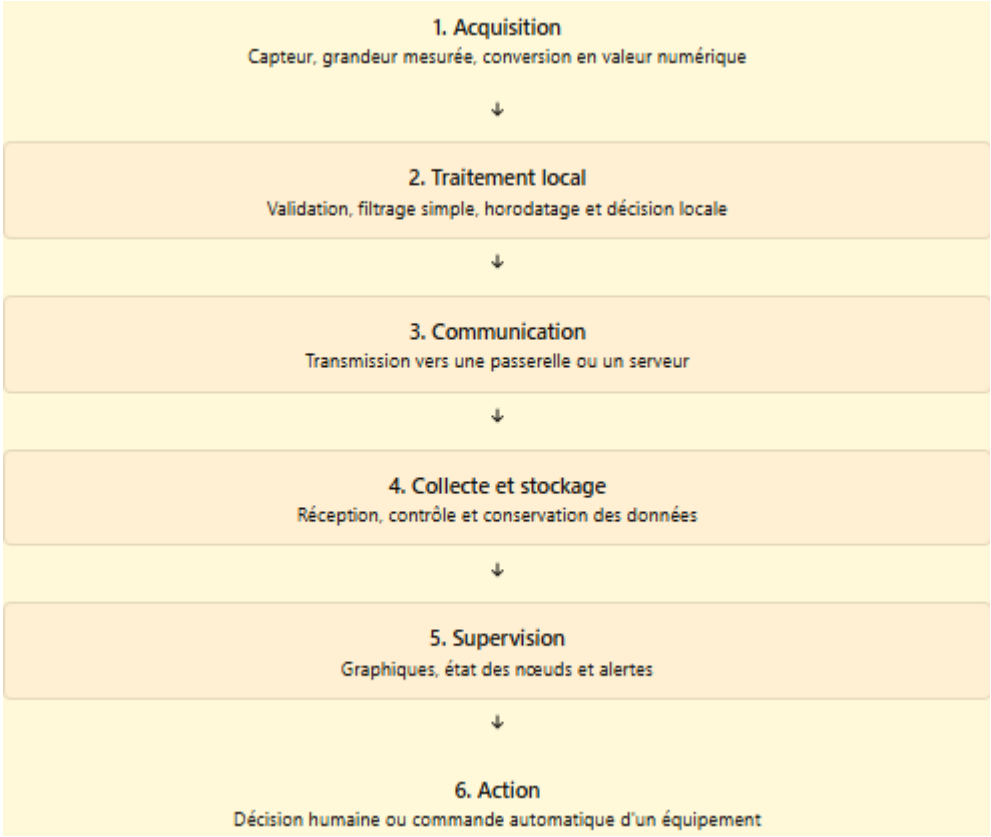
- La conception d'un système IoT commence par un problème concret, et non par le choix immédiat d'un capteur ou d'une carte électronique.
- Prenons l'exemple d'un dispositif de surveillance de l'humidité du sol pour une exploitation agricole au Bénin.
- Le besoin général est : savoir quand une parcelle doit être irriguée, sans exiger une présence permanente de l'agriculteur sur place.
- Il faut transformer ce besoin en exigences vérifiables.

Type d'exigence	Exemple
Fonctionnelle	Mesurer l'humidité du sol et transmettre les mesures
Fonctionnelle	Signaler une humidité inférieure à un seuil
Temporelle	Réaliser une mesure toutes les 5 minutes
Fiabilité	Enregistrer les mesures si la connexion est interrompue
Énergétique	Limiter les transmissions inutiles
Sécurité	Empêcher un appareil non autorisé d'envoyer de fausses mesures
Exploitation	Afficher l'évolution de l'humidité sur un tableau de bord
Maintenance	Permettre de remplacer un nœud défaillant

Une exigence doit, autant que possible, pouvoir être vérifiée par un test. « Le système doit être rapide » est trop vague. « Une alerte doit être affichée moins de 5 secondes après la réception d'une mesure critique par le serveur » est vérifiable.

Décomposition fonctionnelle d'un système IoT

Un système IoT peut être décomposé en six fonctions.



Ces blocs sont logiques : ils ne correspondent pas nécessairement à six appareils différents. Sur un petit système, la collecte, le stockage et la supervision peuvent être assurés par le même ordinateur.

Architecture logicielle

Elle décrit les programmes et les services :

- programme embarqué du nœud ;
- format du message ;
- service de réception ;
- traitement et stockage ;
- tableau de bord ;
- gestion des erreurs et des alertes.

Les deux architectures doivent être cohérentes. Par exemple, si le nœud transmet un identifiant, une mesure et un horodatage, le service de réception doit savoir lire ces trois champs.

Architecture physique et architecture logicielle

Architecture physique

Elle décrit les équipements réels :

- capteurs et alimentation ;
- microcontrôleur ou carte de développement ;
- liaison radio ;
- passerelle éventuelle ;
- serveur ou ordinateur ;
- équipement d'affichage ou d'actionnement.

Le choix d'une architecture dépend notamment de la distance entre les nœuds, de la disponibilité du réseau, du coût des équipements, de l'autonomie recherchée et des conditions de déploiement.

Choix des composants

Le choix ne doit pas être fondé uniquement sur le prix ou la popularité d'une carte.

Critère	Questions à poser
Entrées/sorties	Le composant peut-il lire le capteur choisi ?
Mémoire	Le programme et les données tiennent-ils en mémoire ?
Communication	L'interface disponible répond-elle au besoin de portée et de débit ?
Énergie	La consommation est-elle compatible avec l'autonomie visée ?
Environnement	L'équipement résiste-t-il à la chaleur, à l'humidité et à la poussière ?
Maintenance	Les pièces et les outils de programmation sont-ils accessibles ?
Coût total	Quel est le coût de la carte, de l'alimentation, du boîtier et de la maintenance ?

Conception du format des données

Avant de programmer, il faut définir ce que le nœud transmet.

Exemple de message JSON pour un capteur d'humidité :

```
{
  "node_id": "N01",
  "timestamp": "2026-10-15T10:30:00",
  "soil_moisture": 27.5,
  "battery_v": 3.8,
  "status": "OK"
}
```

Ce format est illustratif. En situation réelle, le format d'horodatage, les unités, le fuseau horaire et les valeurs admissibles doivent être spécifiés. Une valeur d'humidité de 27,5 doit notamment être interprétée selon l'étalonnage du capteur : elle ne représente pas nécessairement 27,5 % d'humidité volumique.

Une spécification de message doit préciser :

1. le nom et le type de chaque champ ;
2. son unité ;
3. les valeurs autorisées ;
4. le comportement en cas de valeur manquante ou invalide ;
5. la règle de détection des messages en double ;
6. le mécanisme d'identification du nœud.

Validation de la conception

Avant de construire le système, on prépare un plan de tests.

Test	Entrée ou situation	Résultat attendu
T1	Humidité normale	Mesure enregistrée
T2	Humidité sous le seuil	Alerte déclenchée
T3	Capteur déconnecté	Erreur signalée, pas de fausse mesure
T4	Connexion interrompue	Mesures conservées localement si cette fonction est prévue
T5	Message mal formé	Message rejeté ou marqué invalide
T6	Nœud inconnu	Message refusé ou placé en quarantaine selon la politique définie

Domaines applicatifs de l'IoT

- **Ville intelligente :** circulation routière intelligente, transports intelligents, collecte des déchets, cartographies diverses (bruit, énergie, etc.).
- **Environnements intelligents :** prédiction des séismes, détection d'incendies, qualité de l'air, etc.
- **Sécurité et gestion des urgences :** radiations, attentats, explosions.
- **Logistique :** aller plus loin que les approches actuelles.
- **Contrôle industriel :** mesure, pronostic et prédiction des pannes, dépannage à distance.
- **Santé :** suivi des paramètres biologiques à distance.
- **Agriculture intelligente, domotique, applications ludiques** etc

Maison intelligente

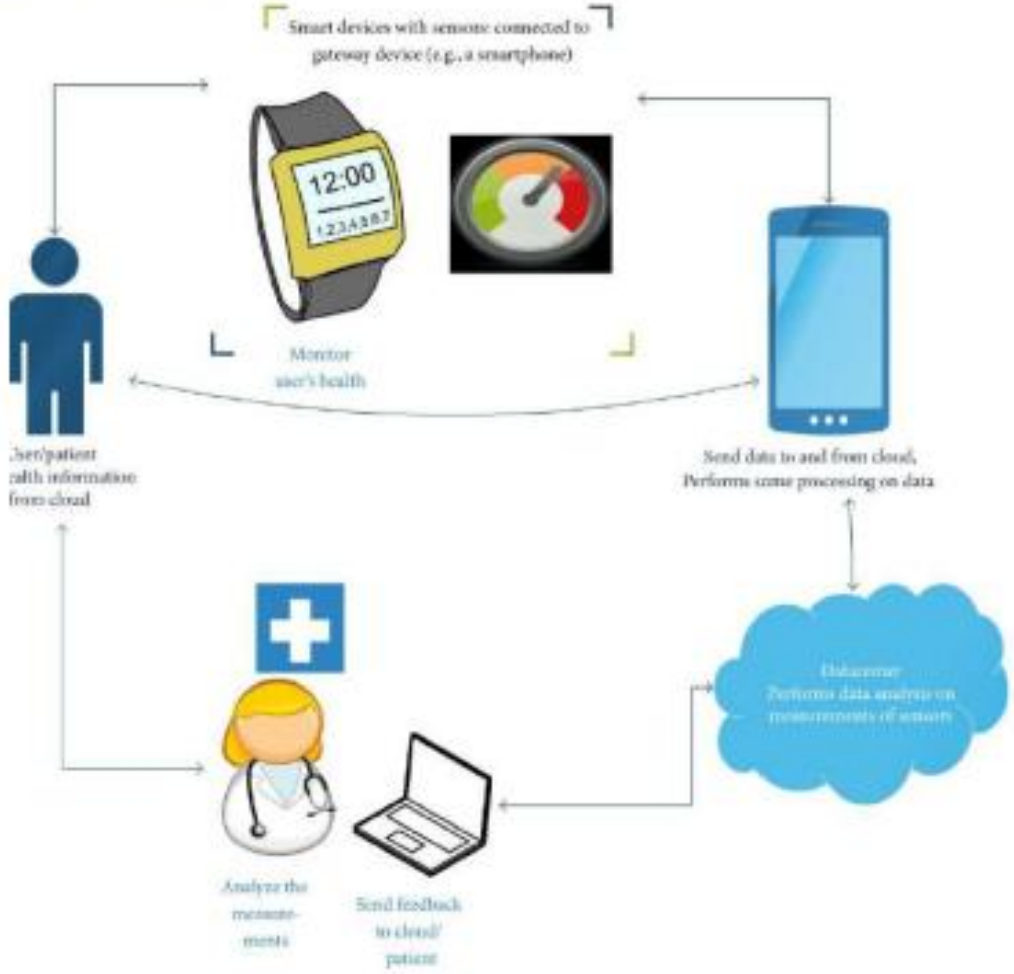


Maison intelligente



Santé et bien être

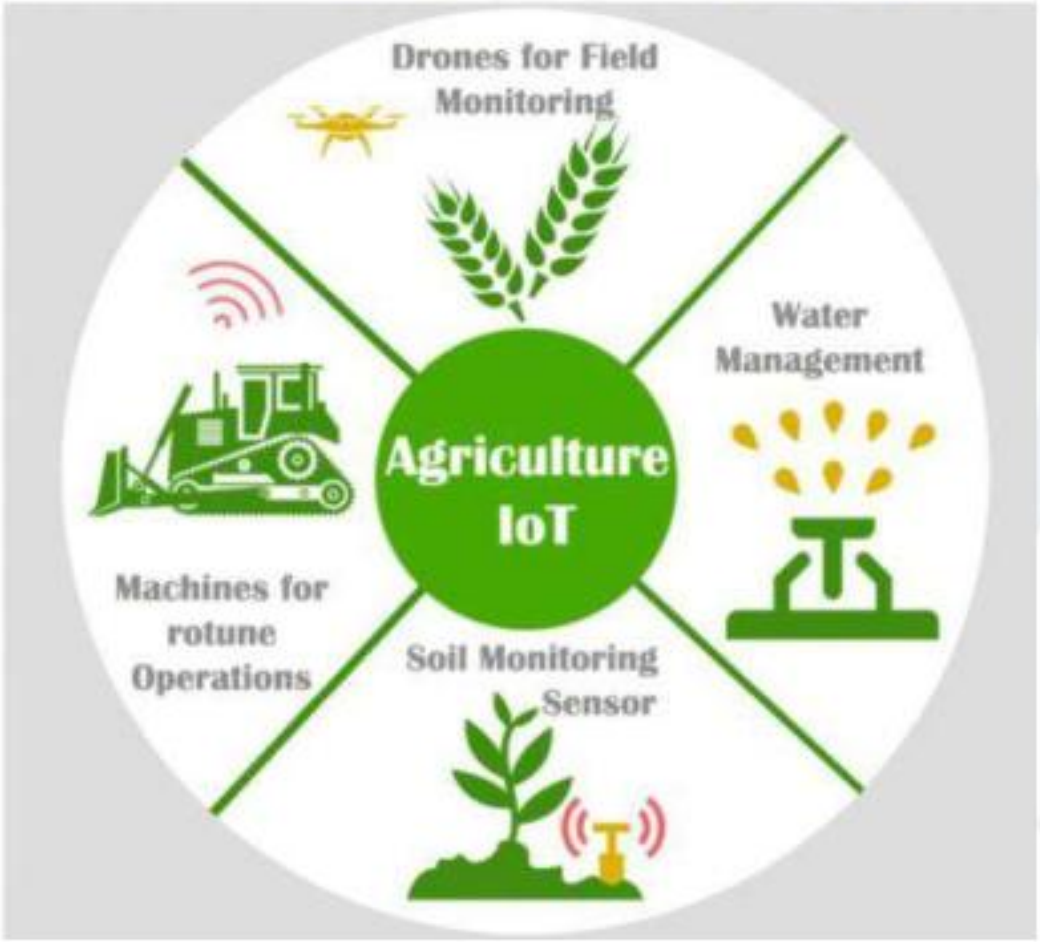




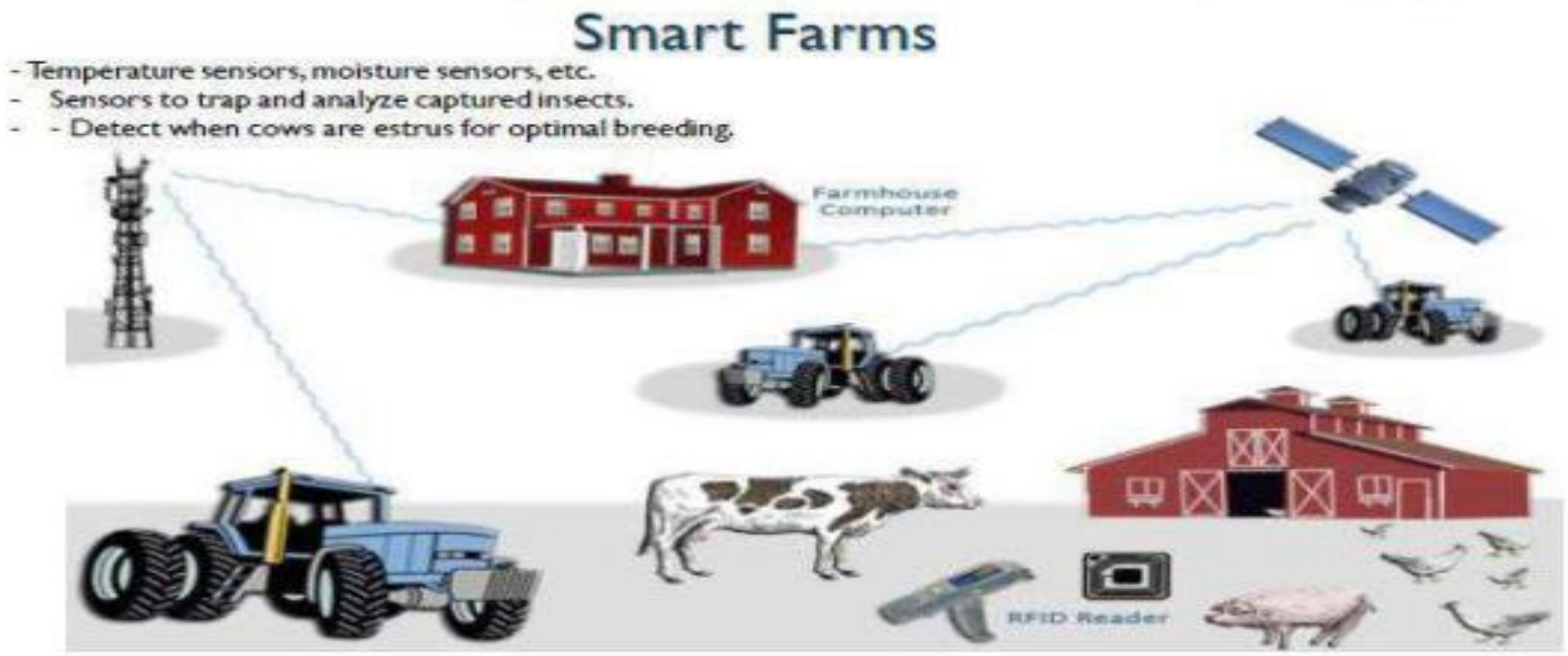
Gestion des déchets dans les vielles intelligentes



Agriculture intelligente



Ferme intelligente

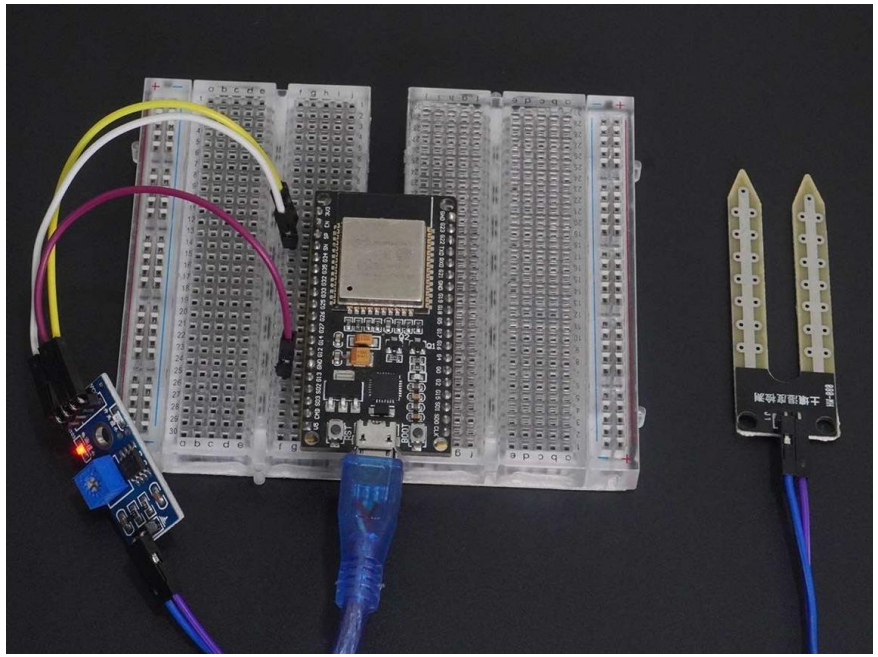


Étude de cas selon les différends domaines

1. Énumérer les exigences fonctionnelles et non fonctionnelles.
2. Proposer les blocs fonctionnels.
3. Définir les champs du message.
4. Identifier trois risques techniques.
5. Proposer cinq tests de validation.

Concevoir l'architecture d'un système IoT du système de surveillance de l'humidité du sol.

1. Définir le besoin et au moins huit exigences.
2. Dessiner l'architecture physique.
3. Dessiner le traitement logiciel d'une mesure.
4. Définir le format d'un message.
5. Construire un tableau de tests.
6. Justifier le choix des composants.



Programmation des nœuds capteurs

Système d'exploitation (RCSF)

Systèmes multitâches		Systèmes basés sur les évênements	Systèmes hybrides	
MANTIS	AmbientRT	**TinyOS (Open Source)**	Contiki	LIMOS

Systèmes basés sur les événements
(= les plus courants)

- *définition des événements gérés par le système*
- *exécution complète et en 1 seule fois de l'action (niv. de priorité)*
- *possibilité de déclenchement d'un événement par un autre*

Dans un capteur les ressources sont limitées (mémoire, puissance de calcul). Il faut donc que le SE soit le plus léger possible.
Deux approches sont utilisées :
– Un SE réduit au minimum (par exemple **Squawk** est une MV java très allégée)
– Un SE modulaire dont on ne charge que les parties utiles (**TinyOS**)
Dans le premier cas on mise sur la puissance du langage associé (java) pour pallier la légèreté du SE. Ce type de solution est plutôt adapté aux futurs capteurs qui auront plus de mémoire et plus de puissance de calcul. Elle est d'ores et déjà très utilisée sur les systèmes mobiles moins contraints (téléphones portables, PDA). Dans le second cas on part de l'idée que dans un SE tout n'est pas utilisé et qu'on peut se contenter, pour une application donnée, d'une partie du SE. Pour permettre cela il faut identifier un noyau minimal du SE puis découper le reste en modules (composants) que le programmeur pourra intégrer à son application.
Il faut un langage de programmation permettant de faire ça facilement, c'est pourquoi TinyOS est lié au langage **nesC** qui définit une application comme un ensemble de composants et de liens entre eux. Les composants du SE sont alors traités comme ceux de l'application et intégrés à la demande.

Quelques SE utilisés sur des capteurs

• Berkeley motes, **TinyOS**

– Mica2, Mica2Dot : microcontrôleur 8-bit, 7.37/4.0 Mhz, 128 KB flash, 4 KB SRAM, radio : CC1000, 512 KB flash externe, 2 batteries AA 3V lithium.

• Intel Mote

– Zeevo : microcontrôleur ARM7, mémoire SRAM et flash, Bluetooth, **TinyOS**

– Mote 2: microcontrôleur 32-bit Xscale PXA 271, **Linux et machine virtuelle Java**.

• Sun SPOT

– microcontrôleur 32-bit ARM920T à 180 Mhz, 512K RAM/4M ROM, radio : ChipCon 2420 et 2.4 GHz IEEE 802.15.4, interface USB, batterie 750 mAh lithium ion, **MV Java Squawk**



TinyOS est un système d'exploitation open-source conçu pour des réseaux de capteurs sans-fil. Il respecte une architecture basée sur une association de composants, réduisant la taille du code nécessaire à sa mise en place. Cela s'inscrit dans le respect des contraintes de mémoires qu'observent les réseaux de capteurs. Pour autant, la bibliothèque de composant de TinyOS est particulièrement complète puisqu'on y retrouve des protocoles réseaux, des pilotes de capteurs et des outils d'acquisition de données. L'ensemble de ces composants peut être utilise tel quel, il peut aussi être adaptée à une application précise. En s'appuyant sur un fonctionnement évènementiel, TinyOS propose à l'utilisateur une gestion très précise de la consommation du capteur et permet de mieux s'adapter à la nature aléatoire de la communication sans fil entre interfaces physiques

TinyOS est écrit en nesC qui un langage dérivé du C qui inclut des notions :
– de composants,
– d'interfaces bidirectionnelles,
– de tâches
– de gestion des interruptions.
TinyOS et nesC sont liés : les applications pour TinyOS doivent être écrites en nesC.

Principes de TinyOS

• Un **composant** est constitué d'au moins un **module** utilisant et fournissant des **interfaces**. S'il contient plusieurs modules les liens entres eux sont décrits par un fichier de **configuration**.

• Une application complète est un composant contenant plusieurs modules liés entre eux dont un module **Main** qui permet de démarrer.

• Le SE offre une centaine de composants que l'on peut utiliser pour écrire des applications. Quand le programme est généré par le compilateur, seuls les composants utilisés (y compris ceux du système) sont présents.

Main est lui-même connecté à certains composants du système (comme l'ordonnanceur par exemple) qui seront donc chargés en mémoire puis lancés par Main

On va décrire dans des fichiers portant le suffixe **.nc** les modules, les interfaces et les configurations.

Une application comporte donc plusieurs de ces fichiers.

Modules et interfaces

• Les **modules** offrent et utilisent des **interfaces**.

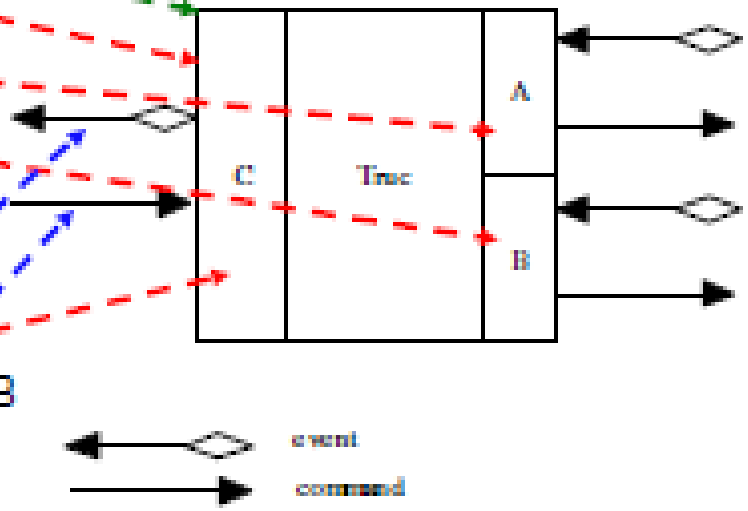
• Une interface déclare un ensemble de fonctions appelées **commands** qui peuvent être utilisées par les autres composants et un ensemble de fonctions appelées **events** qui réagissent à des événements en provenance d'autres composants.

• Toutes ces fonctions doivent être implémentées.

Modules et interfaces

```
module truc {
        provides {
            interface C;
        }
        uses {
                interface A
                interface B
        }

        implementation {
                // code à implémenter :
                // commandes de l'interface C
                // +
                // traitement des événements des interfaces A et B
        }
}
interface C {
        command resultat1 cmd1(paramètres);
        command resultat2 cmd2(paramètres);
        ...
        event resultat3 evt1();
        event resultat3 evt2();
        ...
        }
```



Liaisons

Un composant en nesC est constitué de **modules** et d'une **configuration**.

– Les modules contiennent le code et implémentent une ou plusieurs interfaces.

– Les configurations servent à assembler les composants entre eux en connectant les interfaces utilisées par certains composants à celles fournies par d'autres pour fabriquer de nouveaux composants (composites).

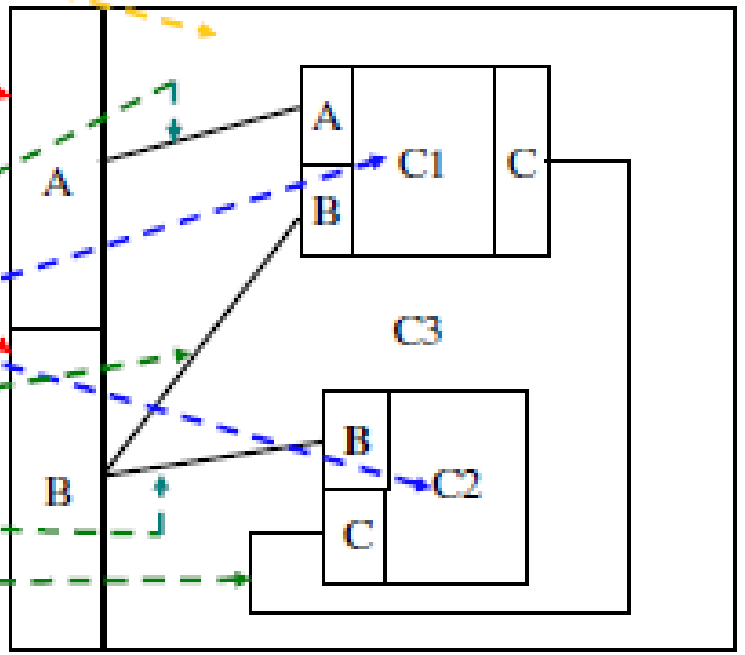
– L'application est elle-même un composant n'offrant aucune interface.

Liaisons :

Fabriquer le composant C3 à partir de C1 et C2.

```
configuration C3 {
    provides {
        interface A;
        interface B
    }

    implementation {
        components C1,C2;
        A = C1.A;
        B = C1.B;
        B = C2.B;
        C1.C -> C2.C;
    }
}
```



Application NESC

• Une application est un assemblage dans lequel apparaît le composant système **Main** qui sert à démarrer.
Elle est décrite par une configuration "de premier niveau".
La partie **configuration** d'un fichier d'une application est vide car elle n'offre et ne requiert aucune interface, la partie implémentation décrit le schéma de câblage de plus haut niveau (assemblage des composants de l'application).
• L'ensemble des configurations permet de savoir quels composants sont utiles pour que l'application fonctionne.
Il suffit de regarder le fichier de configuration de l'application pour savoir quels composants elle utilise (components) puis de faire la même chose récursivement pour chacun de ces composants.

Modèle d'exécution

TinyOS n'exécute qu'une application à la fois.

Il y a deux types de processus d'exécution :

– les **tâches**

– les **pilotes d'interruptions**.

Les tâches s'exécutent l'une après l'autre sans préemption.

Les pilotes d'IT sont exécutés en réponse à une IT mais peuvent préempter les tâches et les autres pilotes d'IT.

Les commandes et les événements qui sont exécutés par un pilote d'IT doivent être définis avec le mot clé **async**.

Problème de temps réel

• Les commandes écrites dans les composants doivent être **de courte durée** car TinyOS ne fait pas de préemption donc si une commande peut durer longtemps le programmeur doit utiliser une **tâche** pour l'exécuter et cette tâche retournera un **événement** pour indiquer qu'elle est terminée.
C'est par exemple le cas des commandes d'envoi sur liaison radio. L'exécution de cette tâche est **différée** puisqu'elle sera mise en file d'attente et exécutée à son tour quand les tâches précédentes seront terminées.
• Les traitements d'événements sont, par nature, **de courte durée**.

Concurrence

Comme les tâches et les pilotes d'IT peuvent être préemptés par des pilotes d'IT, il peut se produire des problèmes de concurrence. Traditionnellement on les résout par l'utilisation de sémaphores. nesC utilise des parties de code atomiques (mot clé **atomic**). Le compilateur nesC détecte les problèmes de concurrence et les indique au programmeur par une erreur de compilation. Lorsqu'il est sûr que le problème ne se pose pas, le programmeur peut demander au compilateur (par le mot clé **norace**) d'accepter une construction apparemment douteuse. Bien entendu il faut utiliser cela avec circonspection.

Tâches

TinyOS connaît les tâches et les pilotes d'IT.

– Les commandes associées aux pilotes d'IT sont déclarées **async** pour pouvoir être exécutées en préemption des autres => elles doivent être rapides.

– Les tâches représentent le reste de l'application et peuvent durer longtemps dans ce cas, la règle est de créer une tâche qui retourne un signal lorsqu'elle est terminée.

– Les tâches sont mises dans une file d'attente et exécutées l'une après l'autre sans temps partagé. Une tâche est déclarée par : task void nomdetache() { ... } elle n'a pas de paramètres et ne retourne rien.

Elle sera mise dans la file d'attente des tâches à exécuter par : post nomdetache();

Le lancement de cette tâche peut se faire dans n'importe quel code (commande ou événement) du composant.

Bloc atomic

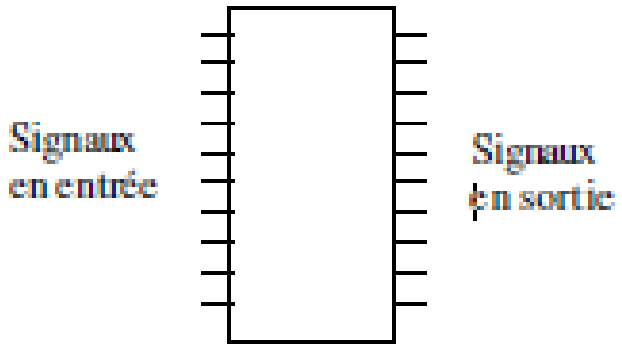
Le bloc **atomic** permet de définir du code qui ne sera pas interrompu par un pilote d'IT :

```
atomic {
for (i=0; i < size; i++) sum = sum + rdata[i];
}
```

Toute la boucle ci-dessus est exécutée **sans interruption** possible. Il ne faut pas en abuser et y mettre le minimum nécessaire sinon les IT ne seront pas prises en compte à temps.

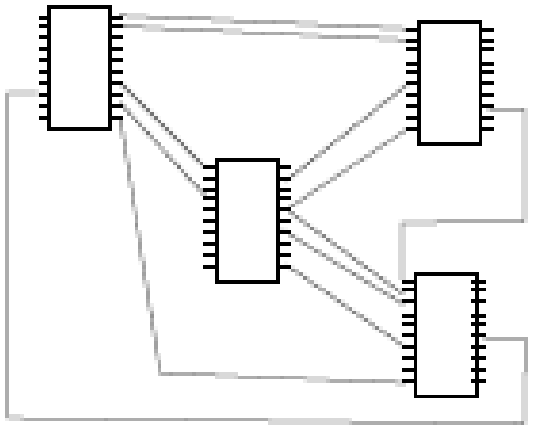
Application en nesC

• nesC est lié à TinyOs et permet de créer des applications comme des composants incluant les composants du SE utiles et seulement ceux-là.
• Une application consiste en des composants reliés entre eux exactement comme un montage électronique.
• Un composant électronique a des entrées (des signaux qu'il utilise) et des sorties (des signaux qu'il fournit) :

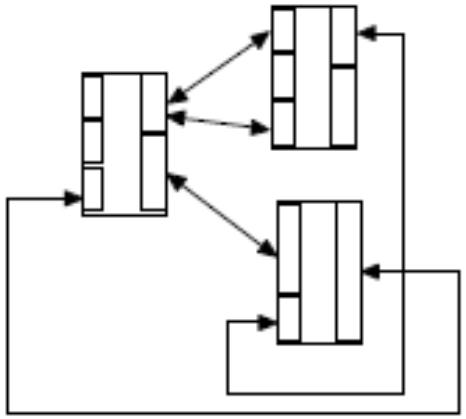


On fera de même pour un composant logiciel. Les signaux en entrée sont utilisés par ce composant (donc fournis par d'autres) tandis que les signaux en sortie sont fournis par ce composant (donc utilisées par d'autres).

On va ensuite interconnecter ces composants entre eux.
En électronique on a un schéma de câblage du type :



Il faut savoir quelles pattes d'un composant relier à quelles autres
Pour les composants logiciels on va constituer les pattes (commandes/événements) en groupes (**interfaces**) et relier directement des groupes entres eux.
Toutes les pattes d'un groupe sont reliées à toutes les pattes d'un autre groupe (leur nom permet de savoir lesquelles sont reliées auxquelles).
Avec nesC on distingue dans une interface les **commandes** (sorties) et les **événements** (entrées). Les liaisons établies entre interfaces sont donc bidirectionnelles car certaines pattes sont des entrées et d'autres des sorties.
Les **commandes** seront utilisées par un appel explicite (comme un appel de fonction) tandis que les **événements** provoqueront automatiquement l'exécution du code approprié dans le composant qui les reçoit.



Les différences essentielles entre un assemblage de composants électroniques et un assemblage nesC sont les suivantes :
– nesC offre des commandes et des événements, un composant électronique ne traite que des événements (réagit à un signal d'entrée) et produit des événements (signal de sortie)
– Les composants nesC sont du code exécuté par un processeur unique => il n'y a aucun parallélisme tandis que les composants électroniques fonctionnent en parallélisme total (entre composants et souvent dans le composant lui-même).

Un petit exemple simple

On va prendre comme exemple un petit programme qui fait clignoter une LED sur un capteur. Il est composé d'un module (ClignoteM.nc) et une configuration (Clignote.nc) qui va lier ce composant aux autres (ici ceux du SE c'est à dire Main, un timer et les leds).

Le fichier de configuration **Clignote.nc** :

```
configuration Clignote {
}

implementation {
  components Main, ClignoteM, SingleTimer,
LedsC;

  Main.StdControl -> ClignoteM.StdControl;
  Main.StdControl -> SingleTimer.StdControl;
  ClignoteM.Timer -> SingleTimer.Timer;
  ClignoteM.Leds -> LedsC;
}
```

Explications

```
configuration Clignote {
}

implementation {
  components Main, ClignoteM, SingleTimer,
LedsC;

  Main.StdControl -> ClignoteM.StdControl;
  Main.StdControl -> SingleTimer.StdControl;
  ClignoteM.Timer -> SingleTimer.Timer;
  ClignoteM.Leds -> LedsC;
}
```

Au début on a un bloc **configuration** qui ici est vide puisqu'il s'agit d'une application. Ce bloc contient normalement les interfaces fournies et requises par le composant.

Ici le fichier Clignote.nc étant la configuration de premier niveau elle ne requiert ni ne fournit rien. Il en serait différemment s'il s'agissait d'un fichier de configuration de niveau inférieur (composant constitué d'autres composants).

Ensuite vient le bloc **implementation** qui indique les composants utilisés et les liens entre eux. Ici on indique que les composants utilisés sont : Main, ClignoteM, SingleTimer et LedsC.

Le composant **Main** est exécuté en premier par TinyOS (c'est comme le main en C) => toute application doit avoir un composant Main dans sa configuration sinon ça ne démarre pas.

Main utilise une interface appelée **StdControl** qui contient 3 commandes : init, start et stop :

```
interface StdControl {
  command result_t init();
  command result_t start();
  command result_t stop();
}
```

```
configuration Clignote {
}

implementation {
  components Main, ClignoteM, SingleTimer, LedsC;

  Main.StdControl -> ClignoteM.StdControl;
  Main.StdControl -> SingleTimer.StdControl;
  ClignoteM.Timer -> SingleTimer.Timer;
  ClignoteM.Leds -> LedsC;
}
```

Ces commandes sont utilisées par le système pour initialiser, lancer et arrêter le composant. La commande init n'est exécutée qu'une seule fois alors que les autres peuvent l'être de multiples fois.

```
configuration Clignote {
}

implementation {
  components Main, ClignoteM, SingleTimer,
LedsC;

  Main.StdControl -> ClignoteM.StdControl;
  Main.StdControl -> SingleTimer.StdControl;
  ClignoteM.Timer -> SingleTimer.Timer;
  ClignoteM.Leds -> LedsC;
}
```

La ligne : Main.StdControl -> ClignoteM.StdControl;

indique que l'on relie l'interface StdControl du composant main à celle du composant Clignote. Ceci signifie que les appels à init start et stop de Main provoqueront des appels à init, start et stop de Clignote.

La flèche doit être comprise comme suit : le composant à gauche **utilise** une interface que celui de droite **fournit**. Donc ici on indique que l'interface StdControl utilisée par le composant Main lui est en fait fournie par le composant Clignote ce qui correspond bien au fait que les commandes init, start et stop utilisées par Main lui sont fournies par Clignote.

On a la même chose entre Main et le composant SingleTimer par la ligne :
Main.StdControl -> SingleTimer.StdControl;
Donc le init de Main appelera celui de Clignote ET celui de SimpleTimer.

La ligne ClignoteM.Timer -> SingleTimer.Timer;
indique que l'interface Timer utilisée par Clignote est celle fournie par SingleTimer qui est un composant de TinyOS. Ainsi dans Clignote on pourra appeler les commandes de SimpleTimer et réagir à ses événements.

De même la ligne : ClignoteM.Leds -> LedsC;
indique que l'interface Leds utilisée par Clignote lui est fournie par LedsC qui est aussi un composant de TinyOS. Ainsi dans Clignote on pourra appeler les commandes de LedsC (LedsC ne produit pas d'événements).

Remarque : comme on n'utilise ici que des interfaces déjà connues de nesC (StdControl, Timer et Leds) on n'a pas a en écrire les fichiers. Si on avait créé un composant avec ses propres interfaces on les aurait décrites dans des fichiers .nc portant le nom de l'interface.

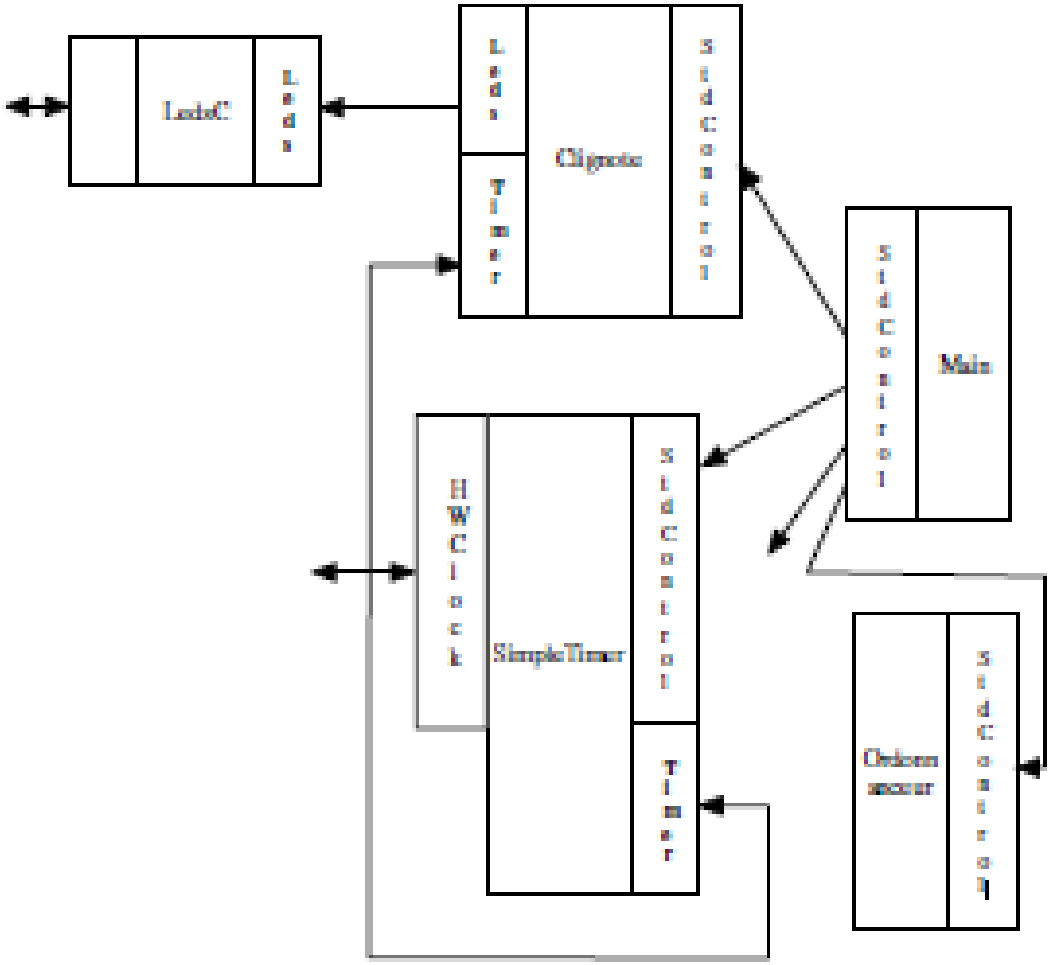
```
configuration Clignote {
}

implementation {
  components Main, ClignoteM, SingleTimer,
LedsC;

  Main.StdControl -> ClignoteM.StdControl;
  Main.StdControl -> SingleTimer.StdControl;
  ClignoteM.Timer -> SingleTimer.Timer;
  ClignoteM.Leds -> LedsC;
}
```

L'application Clignote

On arrive au schéma de câblage suivant :



Syntaxe

• On appelle une commande d'une interface par :

call nom_interface.nom_commande(paramètres)

• On signale un événement par :

signal nom_interface.nom_événement(paramètres)

dans ce cas les paramètre seront transmis à la fonction qui est associée à ce signal.

Dans l'exemple de Clignote l'événement du timer n'a pas de paramètres.

Le fichier du module **ClignoteM.nc**

```
module ClignoteM {

    provides {
      interface StdControl;
    }
    uses {
      interface Timer;
      interface Leds;
    }

    implementation {
      bool etat;
      command result_t StdControl.init() {
        call Leds.init();
        return SUCCESS;
        }

      command result_t StdControl.start() {
        return call Timer.start(TIMER_REPEAT, 1000) ;
        }

      command result_t StdControl.stop() {
        return call Timer.stop();
        }

      event result_t Timer.fired() {
        etat = ! etat;
        if (etat) call Leds.redOn();
        else call Leds.redOff();
        return SUCCESS;
        }
    }
}
```

La partie **provides** indique les interfaces fournies par ce composant.
On y retrouve StdControl puisque ce composant la fournit à Main.

La partie **uses** indique les interfaces utilisées par ce composant.
On y retrouve Timer (fournie par le composant SimpleTimer) et Leds (fournie par le composant LedsC).

```
module ClignoteM {

    provides {
      interface StdControl;
    }
    uses {
      interface Timer;
      interface Leds;
    }

    implementation {
      bool etat;
      command result_t StdControl.init() {
        call Leds.init();
        return SUCCESS;
        }

      command result_t StdControl.start() {
        return call Timer.start(TIMER_REPEAT, 1000) ;
        }

      command result_t StdControl.stop() {
        return call Timer.stop();
        }

      event result_t Timer.fired() {
        etat = ! etat;
        if (etat) call Leds.redOn();
        else call Leds.redOff();
        return SUCCESS;
        }
    }
}
```

La partie **implementation** de Clignote va contenir les codes pour les commandes et la prise en compte des événements.
On y trouve donc les 3 commandes de l'interface fournie StdControl et celle de l'événement du SimpleTimer (fire).

- La commande init se limite à appeler init de LedsC.
- La commande start lance le timer en mode continu pour une durée répétitive de 1 sec.
- La commande stop arrête le timer.
- L'événement fire provenant du timer se limite à basculer la led rouge (on/off) en utilisant les commandes redOn et redOff de LedsC.

```
module ClignoteM {

    provides {
      interface StdControl;
    }
    uses {
      interface Timer;
      interface Leds;
    }

  implementation {
    bool etat;
    command result_t StdControl.init() {
      call Leds.init();
      return SUCCESS;
      }

    command result_t StdControl.start() {
      return call Timer.start(TIMER_REPEAT, 1000);
      }

    command result_t StdControl.stop() {
      return call Timer.stop();
      }

    event result_t Timer.fired() {
      etat = ! etat;
      if (etat) call Leds.redOn();
      else call Leds.redOff();
      return SUCCESS;
      }
  }
}
```

L'interface Leds fournit redOn, redOff, redToggle, greenOn …. qui peuvent être appelées par Clignote pour piloter les LEDs.

L'interface Timer contient les commandes et événements suivants :

```
interface Timer {
  command result_t start(char type,
                         uint32_t interval);
  command result_t stop();
  event result_t fired();
}
```

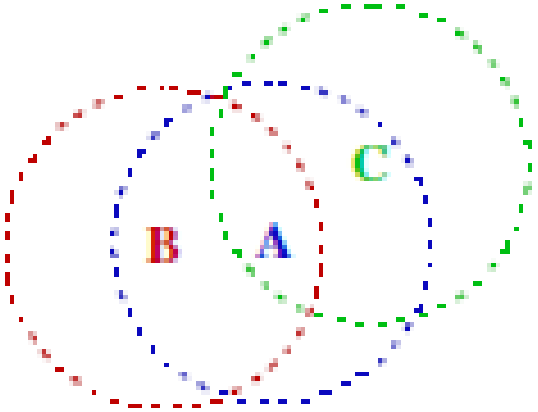
start et stop sont des commandes qui seront appelées par Clignote. En revanche fired est un événement càd fournit un signal et n'est pas appelée de façon explicite par clignote. Ce signal étant relié au composant Clignote, il déclenchera automatiquement l'exécution du code prévu à cet effet dans Clignote.

Accès au medium (MAC)

• ***Le premier problème*** est que pour économiser l'énergie
les capteurs sont mis en veille dès qu'ils n'ont plus d'activité. Lorsqu'un noeud émet il est fort possible que le récepteur ou certains des récepteurs soient en veille

• ***Le second problème*** est que le réseau de capteurs a une topologie quelconque. Les réseaux habituels ont des topologies connues et souvent régulières (bus, anneau …). De plus cette topologie n'est en général pas
connue à l'avance. Enfin, la puissance du signal radio décroît comme le carré de la distance et les capteurs ont des portées de quelques dizaines de mètres.

Dans un réseau classique si B et C reçoivent les messages de A alors B reçoit ceux de C (ils sont sur le même sous-réseau). De plus, selon la topologie (par exemple en étoile) quand A et B dialoguent C ne reçoit rien tandis que sur un bus C reçoit tout et l'ignore.
Dans un réseau de capteurs tout message émis peut être capté par tout noeud à portée (diffusion) mais la visibilité des noeuds est différente, par exemple :



B et C reçoivent les messages de A mais B ne reçoit pas ceux de C.
Conséquence : quand A et B dialoguent C reçoit ce que A émet mais pas ce que B répond. Si C émet en même temps il perturbe A mais pas B donc B continue à recevoir les messages de A mais A ne reçoit plus les réponses de B.

Problème des noeuds en veille

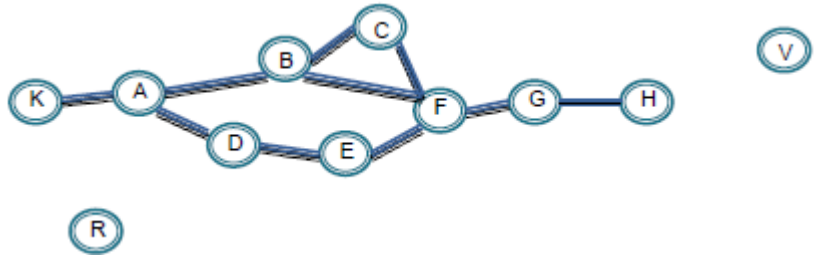
Pour résoudre le problème des noeuds récepteurs en veille on peut utiliser un système de **préambule** càd que le noeud émet une trame sans informations utiles qui sert à alerter les autres noeuds.
Chaque noeud en veille se réveille à intervalles réguliers pour écouter le médium. Il suffit que le préambule dure suffisamment longtemps pour que tous les noeuds se réveillent pendant ce temps.
A la fin du préambule le noeud peut émettre ses infos, il est sûr que le ou les récepteurs sont à l'écoute.
En retour il reçoit un **acquittement**.
Exemple : si un noeud met 1ms à se réveiller et passer en écoute et que le préambule dure 1s, lorsqu'il n'y pas de transmission les noeuds peuvent rester en veille 99,9% du temps.
Un préambule trop court => se réveiller souvent (et souvent pour rien)
Un préambule trop long => perte d'énergie pour émettre
On trouve un compromis par une étude statistique en prenant une durée de préambule de l'ordre de 4 à 6 fois la durée d'une trame.

Routage et simulation des réseaux de capteurs sans fil

Protocoles de routage

Concepts

- Protocoles proactifs : stockage au niveau de chaque mobile d'un chemin vers tous les autres (construction initiale d'une table de routage)
- Protocoles réactifs : construction de la table de routage au gré des communications
- Protocoles hybrides : combinaison proactifs/réactifs (déf. de zones de fonctionnement proactif autour de chaque noeud.)



Les approches de type TCP/IP supposent que chaque machine joue un rôle spécial et que l'on s'adresse à elle pour un service particulier.
Ce n'est pas le cas des réseaux de capteurs.
En effet, un capteur est :
– soit utilisé pour les données qu'il possède
– soit pour sa localisation.
On distingue 3 catégories de protocoles de routage :
– Routage à plat : tous les noeuds jouent un rôle identique
– Routage hiérarchique : les noeuds ont des rôles différents
– Routage basé sur la localisation : la route est calculée en fonction de la position des noeuds
Par ailleurs la façon de trouver une route peut elle-même être divisée en 3 catégories :
– Proactive : les routes sont calculées avant d'être utilisées
– Réactives : les routes sont calculées à la volée
– Hybrides : on mélange les deux méthodes précédentes

Lorsqu'un message circule sur une route, il est possible de lui agréger des données récupérées sur son passage de façon à constituer une information complète.
Si la route n'a pas été prévue dans ce but il se peut qu'à l'arrivée l'information ne contienne pas toutes les données qui auraient pu y être agrégées mais seulement celles qui ont été glanées sur la route.
On peut vouloir mettre en place des protocoles de routage qui trouvent des routes passant par tous les points où il y a une partie d'info à récupérer.

Protocoles de routage à plat

Inondation et inondation aléatoire

- **Inondation** (flooding) : Chaque noeud envoie l'info à tous ses voisins sauf celui d'où il l'a reçue.
- **Inondation aléatoire** (gossiping) : Chaque noeud envoie l'info à certains de ses voisins choisis au hasard y compris à celui qui la lui a envoyée. Il faut qu'un noeud puisse renvoyer l'info à son émetteur pour qu'elle puisse arriver partout. En effet, lorsqu'il la reçoit pour la deuxième fois, l'émetteur peut choisir d'autres voisins au hasard et par conséquent faire en sorte qu'elle arrive à bon port alors que son premier choix ne le permettait pas.

Protocoles de routage à plat

SPIN (Sensor Protocol for Information via Negociation)

Il s'agit d'une famille de protocoles en 3 étapes qui utilise 3 types de messages.

– Quand un noeud veut envoyer une information, pour éviter de consommer trop d'énergie, il envoie à tous ses voisins un message **ADV** qui contient une description de cette information. Cette description est plus brève que l'information elle-même (par exemple si l'info est une image) et coûte moins cher à diffuser. Le protocole ne définit pas le contenu de cette description, il dépend de l'application.

– Si le voisin n'a pas déjà reçu cette information il répond par un message **REQ**. Il recevra alors cette information dans un message **DATA**.

– Après quoi chacun des nouveaux noeuds détenteurs de l'information recommence ce processus jusqu'à ce que tout le réseau ait pu prendre connaissance de cette information.

Diffusion dirigée

Il s'agit d'établir des communications régulières entre noeuds, bien entendu on peut l'utiliser pour une communication ponctuelle mais c'est un peu lourd dans ce cas.
Une demande prend la forme suivante :
– Type : type d'information demandée
– Fréquence : chaque combien de temps cette information doit être transmise
– Zone : peut être utilisé pour définir, quand c'est possible, la zone géographique (rectangle) dans laquelle l'émetteur de l'information doit se situer
– Date de demande : date à laquelle la demande est faite
– Date d'expiration : date à partir de laquelle la demande n'est plus maintenue
Un noeud qui est intéressé pour recevoir un certain type d'information diffuse une telle demande à tous ses voisins et chaque noeud la rediffuse à tous ses voisins. Même s'il la reçoit plusieurs fois, chaque noeud ne la diffuse qu'une seule fois. Le ou les noeuds qui détiennent l'information demandée la diffusent de la même façon.
Avec ces échanges les noeuds vont déterminer la ou les routes possibles.

Méthode dite de Gradient-Based Routing

On va choisir la route la plus courte en nombre de hops :
chaque noeud détermine sa distance en hops au destinataire final, comme ses voisins ont fait la même chose, le gradient du lien entre 2 noeuds A et B est la différence entre la distance de B au destinataire et celle de A.
A choisira d'envoyer sur le lien de plus fort gradient (celle qui rapproche le plus).
On peut fignoler ce procédé pour tenir compte de l'énergie en augmentant le poids d'un noeud quand son énergie baisse (pour éviter de passer pas là).
Quand 2 liens ont le même poids on en prend un au hasard mais on peut aussi essayer de répartir la charge en faisant en sorte de ne pas choisir le même voisin pour plusieurs routes si le gradient est le même.

Méthode basée sur l'énergie

Au lieu de choisir une route optimale on conserve un ensemble de routes possibles. Puis on choisit la route qui minimise l'énergie. L'énergie consommée par chaque route est calculée lors de la diffusion de la requête. Cette méthode permet d'augmenter la
durée de vie du réseau jusqu'à 40%.

- Après quoi les noeuds (demandeur et répondeurs) utiliseront, pour ces échanges, les routes ainsi déterminées.
- Chaque noeud doit donc maintenir une table pour savoir, pour chaque type de demande, à qui il doit faire passer. Cette table lui permet en outre d'éviter de retransmettre une demande qu'il a déjà transmise.
- Le principe utilisé pour les demandes (diffusion totale) fait que chaque noeud a connaissance de toutes les demandes du réseau.
- Lorsqu'elle est utilisé l'information de zone permet de limiter ça à une partie du réseau c'est-à-dire qu'un noeud qui est hors zone et dont tous les voisins le sont aussi peut oublier cette demande. Attention un noeud hors zone dont un voisin est dans la zone peut servir de relais à la communication, il doit donc conserver la demande.

• Quand une demande est diffusée, tout noeud dans la zone qui obtiendra une information correspondant à cette demande la transmettra par la route optimale au demandeur. Il arrêtera de transmettre quand la date limite sera atteinte.
• Un noeud qui reçoit une telle info consulte sa table de demandes pour savoir si elle correspond à une réponse à une demande ou pas. Dans le premier cas il la fait passer par le chemin optimal tandis que dans le second il la diffuse à tous ses voisins (ce qui correspond à la diffusion totale utilisée avant d'avoir déterminé la route optimale).
Remarque : l'utilisation de la date d'expiration peut permettre de traiter la mobilité. On suppose qu'une route reste utilisable jusqu'à cette date puis on recommencera la recherche de route optimale en émettant une nouvelle demande. Ceci permet aussi de prendre en compte le fait que l'information demandée peut concerner un nouveau noeud par exemple un noeud qui s'est déplacé à l'intérieur de la zone d'intérêt alors qu'il n'y était pas quand la route a été établie.

Diffusion dirigée

• Avantages :

Marche même dans des conditions difficiles ou mouvantes : si une route existe on la trouve. Si le réseau est stable une fois les routes trouvées on minimise les chemins donc les consommations.

Permet de faire de l'agrégation de données en choisissant une route qui passe par les nœuds qui doivent compléter l'information. Ceci permet d'envoyer des requêtes complexes dont la réponse doit être apportée par plusieurs noeuds. Par exemple les protocoles COUGAR ou ACQUIRE (Active QUery forwarding In sensoR nEtworks) permettent, lors de la diffusion de la requête, que les noeuds traversés apportent une partie de la réponse avant de diffuser la requête. Dès que la totalité de la réponse est trouvée, cette réponse revient au demandeur.

• Inconvénient :

la topologie peut faire que certains noeuds servent de relais à beaucoup d'informations (ils sont sur beaucoup de routes optimales) => ils épuisent leur énergie. On aurait pu choisir une route différente (même un peu plus longue) pour certaines infos de façon à répartir la charge.

Routage par rumeur

Part du principe que certains noeuds connaissent des événements que d'autres veulent connaître : quelqu'un demande une info à quelqu'un d'autre mais sans savoir qui ni où il est. Par rapport au protocole précédent celui-ci cherche à éviter la diffusion totale des demandes.

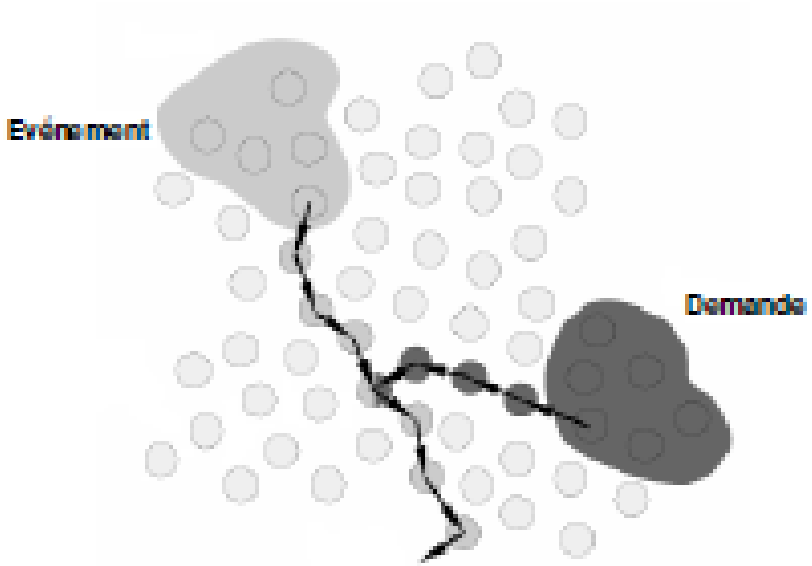
Question : faut-il diffuser les événements ou les demandes ?

S'il y a beaucoup d'événements, en les diffusant on risque de diffuser des infos qui n'intéressent personne. Mais s'il y a beaucoup de demandes il vaut mieux diffuser les événements que les demandes.

Solution retenue : Le noeud origine d'un événement l'envoie à un des ses voisins au hasard et chaque noeud qui les reçoit fait de même. Les événements se promènent ainsi sur le réseau mais avec une durée de vie de type TTL (nombre de hops) ou durée réelle en temps qui dépend de la taille du réseau et du nombre de noeuds. Chaque noeud qui reçoit l'événement connaît le noeud d'origine de cet événement et le chemin qu'il a parcouru et en garde trace.

On diffuse de la même façon les demandes. Quand une demande arrive sur un noeud on regarde si l'on a, sur ce noeud, une trace de l'événement qu'elle demande. Si c'est le cas on sait d'où vient l'événement et on a le chemin pour atteindre son noeud d'origine.

Routage par rumeur



Remarque : Rien ne prouve que les 2 trajectoires (l'événement et la demande) vont se croiser mais en pratique la probabilité est très forte.
On peut toujours prévoir, dans le cas où on n'aboutirait pas, à faire une diffusion totale de la demande ou de l'événement.

Améliorations possibles **:**

1. Quand un événement arrive sur un noeud au lieu de le retransmettre tel quel on peut retransmettre cet événement ET tous ceux qui sont déjà passés sur ce noeud => le noeud suivant reçoit N événements au lieu d'un seul et la probabilité que les routes se croisent augment beaucoup. On peut d'ailleurs faire la même chose pour les demandes.

Enfin on peut mixer ces 2 approches en disant que l'on retransmet l'événement ou la demande accompagnée de TOUS les événements et demandes connues sur ce noeud.

L'inconvénient de cette solution est que la taille des messages croît au fur et à mesure qu'ils avancent.

Améliorations possibles :

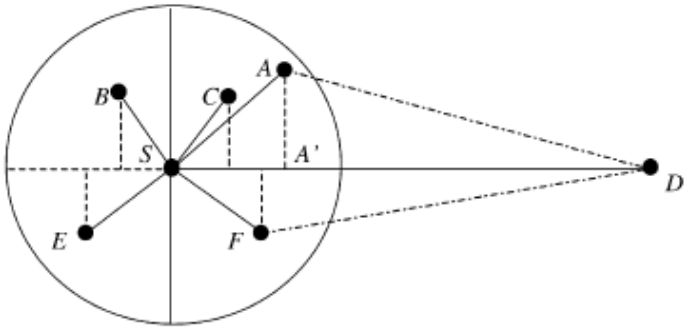
Le noeud origine de l'événement ou de la demande émet un ou plusieurs forking messages. Lorsqu'il arrive sur un noeud le forking message est transmis à un voisin tandis qu'une copie sous forme de message normal est envoyée à un autre voisin => au démarrage on n'a plus qu'une seule trajectoire mais plusieurs qui partent. En fait il est préférable que ce phénomène de duplication ne se produise pas au début de la trajectoire mais après un certain nombre N de hops pour avoir plus de chances de couvrir le réseau. On obtient de bons résultats en prenant pour N environ ¼ de ce qui est pris pour le TTL càd quand la trajectoire déjà parcourue est à ¼ de sa longueur maxi.

Problème possible : un message peut se perdre au cours de sa trajectoire. Pour détecter ça on peut capter le message lorsqu'il est retransmis par le récepteur (A envoie à B qui envoie à C comme A capte B il voit que B a retransmis le message donc qu'il l'a bien reçu sinon il le lui ré-envoie). Mais en raison de l'utilisation de cycles veille/transmission il est possible que A soit en veille lorsque B émet vers C. Une solution raisonnable est d'utiliser un acquittement au moins pour les forking messages.

Protocoles de routage basés sur la localisation

Most Forward Within Radius (MFR)

Principe : Chaque noeud connaît sa position et celle de ses voisins, il peut donc calculer sa distance et celle de ses voisins à la destination. Il envoie l'info à son voisin qui **le rapproche le plus de la destinati**

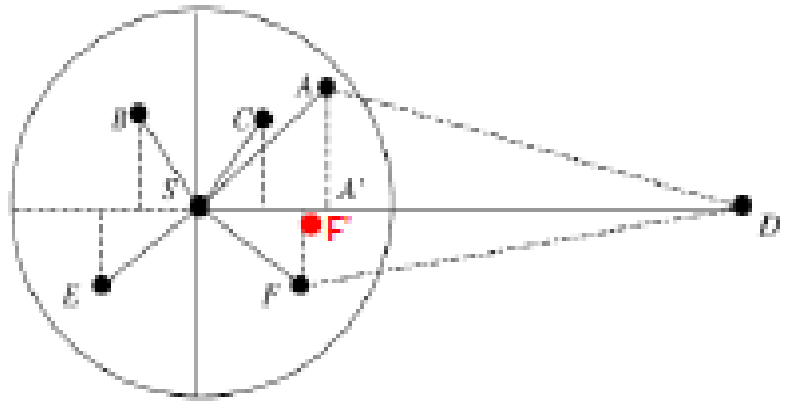


Le noeud S qui veut envoyer à D
choisira A car, sur l'axe SD, le noeud A est celui dont la projection (A') est la plus proche de D.
On considère l'axe SD comme l'axe des x et on cherche le noeud dont la coordonnée en x est la plus grande. Cette méthode peut ne pas aboutir en raison de minima locaux mais elle ne crée pas de boucles.

Greedy

C'est une variante de MFR qui donne en général sensiblement les mêmes résultats.

Principe : Chaque noeud connaît sa position et celle de ses voisins, il peut donc calculer sa distance et celle de ses voisins à la destination. Il envoie l'info à son voisin qui est le **plus proche de la destination finale**. Ce n'est pas équivalent à MFR car le noeud le plus proche de la destination n'est pas forcément celui qui a la coordonnée la plus grande en x sur l'axe SD.

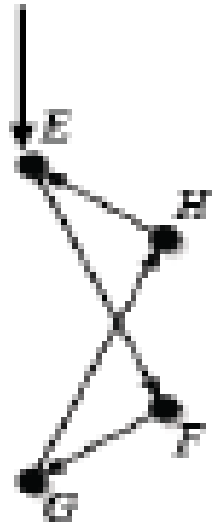


Sur la figure ci-contre le noeud placé en F' est plus près de D que A alors que sa coordonnée en x est inférieure. Donc sur un tel exemple MFR choisirait A tandis que greedy choisirait F'.

Comme MFR, cette méthode peut ne pas aboutir en raison de minima locaux mais elle ne crée par de boucles

Compas (DIR)

L'idée de départ est que chaque noeud S calcule l'angle dont il est le sommet qui est délimité par un voisin V et la destination D. Il choisit le voisin pour lequel cet angle est minimal.
On peut montrer (voir exemple ci-dessous) que cette méthode peut tourner en rond.
Explication du cas ci-contre :
D ne voit pas les noeuds H et F, E et G ne se voient pas.
L'info arrive en E :



• E envoie à F car l'angle DEF est plus petit que DEH et E ne connaît pas G
• F envoie à G car l'angle DFG est plus petit que DFE et que DFH
• G l'envoie à H car l'angle DGH est plus petit que DGF et que G ne voit pas E
• H l'envoie à E car l'angle DHE est plus petit que DHG et que DHF
Et on tourne en rond !

Routage multi chemin

Les méthodes présentées précédemment cherchent un seul chemin. On peut améliorer les performances en utilisant plusieurs chemins. Le choix de l'un des chemins possibles peut être ensuite guidé par différentes raisons.

Dans cette méthode on choisit le chemin sur lequel se trouvent les nœuds disposant de plus d'énergie. A chaque utilisation du chemin on calcule l'énergie restante, lorsqu'elle devient inférieure à celle d'un autre chemin disponible on change de route.

Dans cette méthode on choisit le chemin en fonction d'un calcul concernant la consommation d'énergie que provoque l'utilisation de ce chemin. En effet dans le cas précédent on peut prendre un chemin où l'énergie résiduelle est élevée mais dont l'utilisation la fait rapidementchuter.

Dans cette méthode on découpe le paquet à transmettre en autant de morceaux que de chemins possibles et on envoie chaque morceau par un chemin différent. On peut mettre en place un découpage tel que, s'il manque un seul morceau, on puisse reconstituer le message original.

Méthode basée sur la qualité de service : **Sequential Assignment Routing (SAR)** choisit le chemin en fonction de 3 critères :

- Ressources en énergie
- Qualité de service sur chaque chemin
- Niveau de priorité du message

SAR utilise une somme pondérée de ces 3 critères pour faire le choix. Les chemins possibles sont ré-évalués à intervalles réguliers pour la mobilité. C'est très efficace mais lourd car il faut maintenir des tables de routes et d'états sur chaque noeud.

Méthode basée sur le temps réel : **SPEED** choisit le chemin en fonction du délai de transmission entre noeuds voisins. Ce délai est mesuré par le temps mis pour recevoir le ACK après envoi d'un message mais tient aussi compte du taux d'erreurs de transmission entre voisins.

Modules d’interrogation des données

Envoi de requêtes SQL

- opérateurs classiques ou TinyDB

Systèmes utilisés

- TinyDB (module de TinyOS)
- Cougar

Intérêt:

- sélection des données d’intérêt
- réduction du volume de données transmises…

Exemple:

```
SELECT Volume
FROM Sensor1
GROUP BY date
SAMPLE PERIOD
15min
```